

บันทึกการทุน AI และถอดแบบก่อสร้าง (รวม 4 ตอน + ภาคผนวก)

รวบรวมไดอารี่ประจำวัน ที่ 2026-07-21, 2026-07-24, 2026-07-28, 2026-07-29 และภาคผนวก A-B

รวมทั้ง 4 ไฟล์ `*(teach mk).md` ในไดอารี่นี้ (2026-07-21, 2026-07-24, 2026-07-28, 2026-07-29) ไว้ในที่เดียว เรียงตามลำดับเวลาให้อ่านรวดเดียวจบได้ — เนื้อหาแต่ละตอนคัดลอกมาจากไฟล์ต้นฉบับแบบคำต่อคำ (ไม่ได้สรุปย่อ) ไฟล์ต้นฉบับทั้ง 4 ยังอยู่ครบตามกติกาไดอารี่ (ห้ามลบของเก่า) — ถ้าอยากอ่านทีละวันแยกกันแบบ diary จริง ให้เปิดไฟล์เดิม

เพิ่มจากของเดิม 2 ภาคผนวกท้ายเล่ม ตามที่เมะขามขอเพิ่ม:

- ภาคผนวก A — ไฟล์ทุกไฟล์ใน `tune_ai/t01/` ทำหน้าที่อะไรบ้าง
- ภาคผนวก B — prompt จริงที่ส่งไปหา Qwen ใน t01 (ทั้ง 2 ตัว: classify + extract)

สารบัญ

- ตอนที่ 1 (2026-07-21) — เข้าการดจ่อครั้งแรก, LoRA/MoE/OOM, ทุน t01 สำเร็จ, GGUF อ่านภาพไม่ได้
- ตอนที่ 2 (2026-07-24) — "Phase 0" เช็คก่อนเข้ารอบสอง, เจอบั๊ก MAX_LENGTH
- ตอนที่ 3 (2026-07-28) — t02 ลงมือเข้าจริง (Qwen3-VL เทียบกับ Qwen3.6), เจอบั๊ก resize เจียน
- ตอนที่ 4 (2026-07-29) — ใช้ t01 จริงครั้งแรกแล้วพัง, สงสัยว่า LoRA ไม่อยู่ใน GGUF
- ภาคผนวก A — ไฟล์ทุกไฟล์ใน t01 ทำหน้าที่อะไร
- ภาคผนวก B — prompt จริงที่ส่งไปหา Qwen

ตอนที่ 1 (2026-07-21) — ทุกอย่างที่เกิดขึ้นวันนี้ คืออะไร ทำไมถึงเป็นแบบนี้

ไฟล์นี้ไม่ใช่ log ปกติ (ดู `2026-07-21.md` สำหรับ log จริง) — ไฟล์นี้ตั้งใจเขียนให้อ่านเข้าใจได้ แม้ไม่มีพื้นฐาน AI/programming มาก่อนเลย สมมติว่าคุณไม่รู้อะไรเกี่ยวกับเรื่องนี้เลยสักนิด

1. ภาพรวม: วันนี้ทำอะไรอยู่ (เส้นทางเดินทั้งหมด)

```
ข้อมูลที่ตรวจแก้ไขแล้ว (json_แก้ไขแล้ว/)
↓
เข้าคอมพิวเตอร์แรงๆ ที่มี "การดจ่อ" พิเศษ (Vast.ai)
↓
ต่อเข้าเครื่องนั้นจากคอมเรา (SSH)
↓
ส่งข้อมูลไปเก็บในเครื่องเช่า (scp)
↓
เช็คเครื่องพร้อมจริง (verify_env.py)
↓
ทดสอบสอนโมเดลแบบสั้นๆ ก่อน (TEST_STEPS=5)
↓
สอนโมเดลเต็มรูปแบบ (train_qwen36.py, 3 รอบ)
↓
วัดผลว่าโมเดลเก่งขึ้นไหม (eval_fields.py)
```

เป้าหมายสุดท้าย: ให้ AI (โมเดลชื่อ Qwen3.6) อ่านแบบก่อสร้างไทยแล้วสรุปข้อมูลหลัก/दान/เสาออกมาเป็นตัวเลขได้แม่นยำกว่าตอนที่ยังไม่ได้สอนมัน

2. ทำไมต้อง "เช่า" คอมพิวเตอร์ (GPU) แทนใช้คอมตัวเอง

GPU (การ์ดจอ) คือชิ้นส่วนคอมพิวเตอร์ที่คำนวณตัวเลขจำนวนมากพร้อมกันได้เร็วมาก — ปกติใช้เล่นเกม/ตัดต่อวิดีโอ แต่ AI ก็ต้องใช้มันคำนวณเหมือนกัน (เพราะ AI คือการคูณ-บวกตัวเลขนับล้านๆ ครั้งซ้ำๆ)

โมเดล AI ที่เราใช้ (Qwen3.6) มีขนาดใหญ่มาก — มี "พารามิเตอร์" (ตัวเลขที่โมเดลใช้ตัดสินใจ) ถึง **35,000 ล้านตัว** การ์ดจอในคอมทั่วไป (เช่น ที่คุณมี RTX 5060 มี VRAM 8GB) เก็บโมเดลขนาดนี้ไม่ได้เลย ต้องใช้การ์ดที่มี **VRAM (หน่วยความจำของการ์ดจอ) อย่างน้อย 74GB** — การระดับนี้ราคาซื้อขายเป็นแสนๆ บาท จึงเลือก "เช่า" เป็นชั่วโมง แทน (เหมือนเช่ารถ ไม่ต้องซื้อขาด) ผ่านเว็บ Vast.ai — จ่ายแค่ตอนใช้งานจริง (\$1.056/ชม. สำหรับการ์ดที่เช่าวันนี้)

3. SSH คืออะไร ทำไมไม่ใช้เว็บ (Jupyter)

SSH คือวิธี "ต่อสายเข้าไปควบคุมคอมพิวเตอร์อื่นจากระยะไกล" ผ่าน terminal (หน้าจอสีดำพิมพ์คำสั่ง) — เหมือนเราสามารถนั่งอยู่บ้าน แต่พิมพ์คำสั่งควบคุมคอมที่อยู่อีกประเทศหนึ่งได้เลย

ตอนแรกลองต่อผ่าน **Jupyter** (หน้าเว็บที่ให้พิมพ์โค้ดได้) แต่เจอปัญหา "เปิดไม่ติด" ซ้ำๆ หลายรอบ พอไปหาข้อมูลพบว่าคนที่เทรน AI จริงจังทั่วโลกใช้ SSH กันเป็นหลัก เพราะเสถียรมากกว่า เร็วกว่า และมี **tmux** ติดมาด้วย (โปรแกรมที่ทำให้เปิดหน้าต่างได้โดยงานยังทำงานต่อในเครื่องเช่า ไม่ต้องนั่งเฝ้าหน้าจอตลอด)

กุญแจ SSH (SSH key): แทนที่จะพิมพ์รหัสผ่านทุกครั้ง เราสร้าง "กุญแจดิจิทัล" ขึ้นมา 2 ดอก — ดอกหนึ่ง (private) เก็บในคอมคุณ อีกดอก (public) เอาไปฝากไว้กับ Vast.ai เวลาต่อเข้าเครื่องเช่า ทั้งสองดอกจะเช็คกันเองว่าตรงกันไหม ปลอดภัยกว่ารหัสผ่าน และทำครั้งเดียวใช้ได้ตลอดไป

4. Unsloth คืออะไร — ทำไมไม่ต้อง "เช่า" มันเหมือนเว็บไซต์

คำถามนี้ดีมาก เพราะเป็นความเข้าใจผิดที่พบบ่อย: **Unsloth ไม่ใช่เว็บไซต์หรือแอปที่ต้อง login เข้าไปใช้งาน**

Unsloth คือ "**ไลบรารี**" (library) — เปรียบเหมือนหนังสือสูตรอาหารเล่มหนึ่งที่วางอยู่บนชั้น พอเราจะทำอาหาร (เขียนโปรแกรม) เราแค่ "หยิบหนังสือมาเปิดดู" (พิมพ์คำสั่ง `import unsloth` ในโค้ด Python) แล้วใช้สูตรในนั้นได้เลย ไม่ต้องสมัครสมาชิก ไม่ต้อง login ไม่ต้องต่อเน็ตไปหาบริษัท Unsloth เลยด้วยซ้ำ (แค่ตอนติดตั้งครั้งแรกต้องโหลดไฟล์มาเก็บไว้ในเครื่อง เหมือนซื้อหนังสือมาวางบนชั้นครั้งเดียว หลังจากนั้นก็หยิบใช้ได้เรื่อยๆ)

Unsloth ทำหน้าที่อะไร: ทำให้การสอนโมเดล AI (fine-tuning) ใช้ **VRAM น้อยลงและเร็วขึ้น** โดยใช้เทคนิคเฉพาะทาง (เช่นจัดการหน่วยความจำฉลาดขึ้น) เทียบกับถ้าเราเขียนโค้ดสอนโมเดลเองตั้งแต่ต้นแบบไม่มีเครื่องมือช่วยเลย — เหมือนมีผู้ช่วยทำครัวที่รู้เทคนิคลัดทำให้เสร็จเร็วขึ้นและใช้วัตถุดิบน้อยลง

สิ่งที่ต้องมี "**เว็บ/บริการจริง**" คือ **HuggingFace** (ที่เก็บตัวโมเดล Qwen3.6 ไว้ให้ดาวน์โหลด) — อันนี้ต้องต่อเน็ตไปโหลดไฟล์โมเดลมาจริงๆ ครั้งแรก (~70GB) แต่หลังจากนั้นก็เก็บไว้ในเครื่องเช่าแล้ว ไม่ต้องโหลดซ้ำ

5. Fine-tuning (การทูน) คืออะไร

โมเดล AI ที่ดาวน์โหลดมา (Qwen3.6) ถูกสอนมาแล้วจากข้อมูลทั่วไปในอินเทอร์เน็ตนับล้านๆ หน้า — มันเก่งเรื่องทั่วไป แต่ไม่เคยเห็นแบบก่อสร้างไทยของเราเลย ไม่รู้ว่าเราอยากได้คำตอบในรูปแบบไหน (JSON แบบไหน มี field อะไรบ้าง)

Fine-tuning (การทูน) คือการเอาโมเดลที่เก่งอยู่แล้วนี้ มา "**สอนเพิ่มเติมเฉพาะทาง**" ด้วยตัวอย่างของเรา (226 คู่ ภาพแบบ+คำตอบที่ถูกต้อง) ให้มันคุ้นเคยกับงานนี้โดยเฉพาะ — เหมือนหมอที่เรียนแพทย์ทั่วไปมาแล้ว แล้วมาเรียนเฉพาะทางเพิ่มเติมเป็นหมอกกระดูก ไม่ต้องเรียนแพทย์ใหม่ทั้งหมดตั้งแต่ต้น

6. LoRA คืออะไร

โมเดลมี 35,000 ล้านพารามิเตอร์ ถ้าจะ "สอนเพิ่ม" แบบแก้ทุกตัวเลขเลย จะใช้ **VRAM มหาศาลและเสี่ยงทำโมเดลพังความรู้เดิม** (เหมือนเขียนทับหนังสือทั้งเล่มใหม่หมด)

LoRA (ย่อจาก Low-Rank Adaptation) คือเทคนิคที่ไม่แก้พารามิเตอร์เดิมเลยสักตัว แต่สร้าง "ตัวปรับแต่งเสริม" ขนาดเล็กๆ แปะเข้าไปข้างๆ แทน — เหมือนเราไม่เขียนทับหนังสือเล่มเดิม แต่แปะ **โพสติดิ** (post-it) เล็กๆ ไว้ข้างหน้าแต่ละหน้าที่ยากแก่ พอโมเดลจะตอบอะไร มันจะดูทั้งเนื้อหาเดิม + โพสติดิที่แปะไว้ประกอบกัน

LORA_R คือ "ขนาดของโพสติดิ" — ตัวเลขยิ่งสูง (เช่น 64) โพสติดิยิ่งใหญ่ เขียนอะไรได้ละเอียดขึ้น แต่ก็กิน VRAM มากขึ้นด้วย วันนี้ตอนแรกตั้งไว้ 64 แล้วเจอปัญหาหน่วยความจำเต็ม (ดูข้อ 8) เลยลดเหลือ 32 — โพสติดิเล็กลงหน่อย แต่ยังพอเขียนสิ่งที่จำเป็นได้ และงานวิจัยบอกว่าถ้าข้อมูลสอนมีน้อย (226 ตัวอย่าง ถือว่าน้อย) โพสติดิเล็กกว่าจะยิ่งเหมาะสมกว่าด้วยซ้ำ (ไม่จิ้นเสี่ยง "จำแบบผิดๆ" จนเสียของเดิมไป)

ข้อดีอีกอย่าง: พอเทรนเสร็จ เราจะได้ไฟล์ "โพสติดิ" (LoRA adapter) ที่ **เล็กมาก** (ไม่กี่ร้อย MB) แทนที่จะต้องเก็บโมเดลทั้งก้อน 35,000 ล้านพารามิเตอร์ใหม่ทั้งหมด (ที่จะหนักหลายสิบล GB)

7. MoE (Mixture of Experts) คืออะไร — ทำไมเกี่ยวกับปัญหาที่เจอวันนี้

โมเดล Qwen3.6 ที่ใช้ ไม่ได้เป็น "สมองก้อนเดียว" แบบทั่วไป แต่เป็นแบบ **MoE** (มีผู้เชี่ยวชาญย่อยหลายคน) — ในโมเดลนี้มี **256 "ผู้เชี่ยวชาญย่อย" (experts)** ซ่อนอยู่ข้างใน แต่แต่ละครั้งที่ตอบคำถาม โมเดลจะเลือกผู้เชี่ยวชาญไม่กี่คนมาช่วยตอบ (ไม่ใช่ทั้ง 256 คนพร้อมกันทุกครั้ง) — ทำให้แม้โมเดลจะมีขนาดใหญ่มาก (35,000 ล้านพารามิเตอร์) แต่ตอนคำนวณจริงใช้แค่ ~3,000 ล้านตัวต่อครั้ง (เร็วกว่าถ้าต้องใช้ทั้งหมด)

ทำไมเรื่องนี้เกี่ยวกับปัญหาหน่วยความจำเต็มวันนี้: เวลาแปะ "โพสติดิ LoRA" (ข้อ 6) เข้าไปที่ผู้เชี่ยวชาญ — เพราะมีผู้เชี่ยวชาญตั้ง 256 คน โพสติดิเลยต้องแปะซ้ำๆ 256 รอบ (คนละใบ) ทำให้กินหน่วยความจำมากกว่าโมเดลทั่วไปที่ไม่ใช่ MoE มาก นี่คือสาเหตุแท้จริงที่ทำให้หน่วยความจำเต็มวันนี้

8. "หน่วยความจำเต็ม" (CUDA Out of Memory / OOM) คืออะไร

การจำลองมีหน่วยความจำของตัวเอง (VRAM) แยกจาก RAM ปกติของคอม — วันนี้เช่าการ์ดที่มี VRAM 95GB

ตอนสอนโมเดล (เทรน) ต้องเก็บของหลายอย่างพร้อมกันใน VRAM:

- ตัวโมเดลเอง (~70GB สำหรับ 35,000 ล้านพารามิเตอร์)
- "โพสติดิ LoRA" (ข้อ 6-7)
- ค่ากลางระหว่างคำนวณ** (activation memory) — ยิ่งข้อความ/ภาพที่ป้อนเข้ายาว ก็ยิ่งต้องเก็บค่ากลางเยอะ

วันนี้ VRAM 95GB **เกือบเต็มพอดี** (ใช้ไป 94.87GB เหลือแค่ ~100MB) แล้วพอโมเดลต้องการพื้นที่เพิ่มอีกนิดเดียว (100MB) ก็หาไม่ได้แล้ว → error **"CUDA out of memory"** (แปลตรงตัว: การจำลอง [ที่ใช้เทคโนโลยีชื่อ CUDA] หน่วยความจำเต็ม)

เหมือนกระเป๋ากับที่จัดของแน่นเป๊ะจนเหลือช่องว่างนิดเดียว พอมีของอีกชิ้นมาต้องใส่ (แม้จะเล็กนิดเดียว) ก็ยัดไม่เข้าอีกแล้ว

วิธีแก้ที่ลองไปวันนี้ (เรียงจากที่ลองก่อน):

- ปรับการจัดการหน่วยความจำให้ฉลาดขึ้น (ลดพื้นที่ที่ "จองไว้เผื่อ" แต่ไม่ได้ใช้จริง) — **ลองแล้วไม่พอ**
- ลดขนาด "โพสติดิ LoRA" (ข้อ 6, จาก 64 เหลือ 32) — **วิธีนี้ได้ผลจริง** เพราะ MoE มีผู้เชี่ยวชาญ 256 คน (ข้อ 7) การลดขนาดโพสติดิช่วยประหยัดคุณ 256 เท่า

9. ไฟล์ต่างๆ ที่เกี่ยวกับ "การทวน" คืออะไรบ้าง

ไฟล์	คืออะไร
<code>train.jsonl</code>	ชุดตัวอย่างที่ใช้ "สอน" โมเดล (226 คู่ ภาพแบบ+คำตอบที่ถูกต้อง จากบ้าน 01,02,04) — แต่ละบรรทัดคือ 1 ตัวอย่าง
<code>val.jsonl</code>	ชุดตัวอย่างที่ใช้ "สอบ" โมเดล (88 คู่ จากบ้าน 03) — โมเดลไม่เคยเห็นชุดนี้ตอนสอน ใช้เช็คการเรียนรู้จริงหรือแค่ท่องจำ
<code>train_qwen36.py</code>	โปรแกรม (สคริปต์) ที่สั่งให้ "เริ่มสอน" โมเดลจริงๆ
<code>eval_fields.py</code>	โปรแกรมที่สั่งให้ "เอาโมเดลไปสอบ" กับ <code>val.jsonl</code> แล้วสรุปคะแนน
<code>verify_env.py</code>	โปรแกรมเช็คเครื่องเข้าพร้อมสอนหรือยัง (การตรวจ, VRAM, โลบารต่างๆ)
checkpoint (เช่น <code>outputs_qwen36/checkpoint-5/</code>)	"จุดบันทึกความคืบหน้า" ระหว่างสอน — เหมือน save เกมระหว่างเล่น เพื่อไฟดับ/พัง กลางทางจะได้ไม่ต้องเริ่มใหม่หมด
LoRA adapter (<code>outputs_qwen36/lora/</code>)	ไฟล์ผลลัพธ์สุดท้ายหลังสอนเสร็จ — คือ "โพสตอิท" ที่พูดถึงในข้อ 6 (ไฟล์เล็ก ไม่ใช่โมเดลทั้งก้อน) เอาไปปะกับโมเดลต้นฉบับตอนใช้งานจริง

10. วิธีอ่านผลลัพธ์ตอนเทรน/วัดผล — ตัวเลขพวกนี้แปลว่าอะไร

ระหว่างเทรน จะเห็นตัวเลขแบบนี้:

```
{'train_loss': '1.811', 'epoch': '0.177'}
```

- **loss** = "ค่าความผิดพลาด" — ยิ่งตัวเลขต่ำ ยิ่งดี (โมเดลทายคำตอบใกล้เคียงของจริงมากขึ้น) เริ่มต้นจะสูง แล้วค่อยๆ ลดลงเรื่อยๆ ระหว่างเทรน
- **epoch** = "รอบ" — สอนครบทุกตัวอย่างในชุด train 1 รอบ = epoch ที่ 1 (วันนี้ตั้งใจสอน 3 รอบ = `epoch=3`) ตัวเลข 0.177 หมายถึงเทรนไปได้ 17.7% ของ 1 รอบ (เพราะเป็นแค่การทดสอบสั้นๆ 5 ก้าว ไม่ใช่รอบเต็ม)
- **step** = "ก้าว" — 1 ก้าว = ปรับน้ำหนักโมเดล 1 ครั้ง (สอนตัวอย่างไปกลุ่มหนึ่งแล้วปรับ)

ตอนวัดผลกับชุดสอบ (`eval_fields.py`) จะเห็นแบบนี้:

```
JSON valid      : 0/20   (0.0%)
element หาเจอ  : 0/170  (0.0%)
```

- **JSON valid** = จากคำตอบที่โมเดลตอบมา ก็ % ที่เป็น JSON ถูกรูปแบบ (parse ได้) — ก่อนสอน (baseline) ได้ 0% เพราะโมเดลไม่เคยเห็นรูปแบบนี้มาก่อนเลย
- **element หาเจอ** = จากรายการหลัก/คาน/เสาที่ควรจะมีทั้งหมด (170 รายการ ใน 20 ตัวอย่างที่สอบ) โมเดลตอบถูกต้องตรงกันกี่รายการ
- **field exact-match** = ในรายการที่หาเจอแล้ว แต่ละช่องข้อมูล (เช่น ความยาวคาน, ขนาดเหล็ก) ตอบตรงเป๊ะกี่ %

หลักการอ่าน: ก่อนสอน (baseline) ตัวเลขพวกนี้ควรแย่มาก (ใกล้ 0%) — เป็นเรื่องปกติ ไม่ใช่ของพัง เพราะโมเดลไม่เคยเห็นงานนี้มาก่อน เป้าหมายคือหลังสอนเสร็จแล้ว ตัวเลขพวกนี้ต้องดีขึ้นชัดเจน ถ้าวัดหลังสอนแล้วยังแยเท่าเดิม แปลว่าการสอนไม่ได้ผล ต้องกลับไปแก้

11. สรุปคำศัพท์ท้ายเล่ม (เปิดดูเร็วๆ ได้)

คำศัพท์	ความหมายสั้นๆ
GPU / การ์ดจอ	ชิปคำนวณตัวเลขเร็วมาก ใช้ทั้งเล่นเกมและทำ AI
VRAM	หน่วยความจำของการ์ดจอ (แยกจาก RAM ปกติ)
SSH	วิธีต่อควบคุมคอมพิวเตอร์อื่นจากระยะไกลผ่านการพิมพ์คำสั่ง
tmux	โปรแกรมที่ทำให้เปิดหน้าต่างได้โดยงานในเครื่องเข้ายังทำงานต่อ
scp	โปรแกรมคัดลอกไฟล์ผ่าน SSH
Unsloth	ไลบรารี (เครื่องมือ) ช่วยสอนโมเดล AI ให้เร็ว/ประหยัด VRAM ขึ้น ไม่ใช่เว็บ/แอปที่ต้อง login
HuggingFace	เว็บที่เก็บโมเดล AI ให้ดาวน์โหลด
Fine-tuning (การทุน)	สอนโมเดลที่เก่งอยู่แล้วให้ทำงานเฉพาะทางเพิ่ม
LoRA	เทคนิคแปะ "โพสต์อิท" ปรับแต่งเสริม แทนแก้โมเดลทั้งก้อน
LORA_R	ขนาดของโพสต์อิท LoRA (ยิ่งสูงยิ่งละเอียดแต่กิน VRAM มากขึ้น)
MoE (Mixture of Experts)	โมเดลที่มีผู้เชี่ยวชาญย่อยหลายคนข้างใน เลือกใช้บางคนต่อครั้ง
CUDA out of memory (OOM)	VRAM เต็ม ไม่พอให้คำนวณต่อ
loss	ค่าความผิดพลาดของโมเดล ยิ่งต่ำยิ่งดี
epoch	1 รอบสอนครบทุกตัวอย่างในชุด train
step	1 ครั้งของการปรับน้ำหนักโมเดล
checkpoint	จุดบันทึกความคืบหน้าระหว่างเทรน (เหมือน save เกม)
LoRA adapter	ไฟล์ผลลัพธ์สุดท้าย (โพสต์อิทที่สอนเสร็จแล้ว)
baseline	ผลวัดของโมเดลก่อนสอน ใช้เทียบว่าหลังสอนดีขึ้นแค่ไหน

12. เหตุการณ์สืบสน "ทำไม log ไฟล์ที่สั่งไว้ถึงไม่มี" — บทเรียนเรื่อง terminal คำคำสั่ง

หลังทดสอบ 5 step ผ่านแล้ว (ข้อ 8-9) สั่งให้เริ่ม เทรนเต็ม 3 epoch จริง โดยพิมพ์คำสั่งใหม่ต่อเข้าไปในหน้าต่าง terminal เดิมทันที — แต่พอเช็คภายหลังกลับพบว่า ไฟล์ log ของการเทรนเต็ม (`full_train.log`) ไม่มีอยู่จริงเลย ทั้งที่เห็นคำสั่งถูก "พิมพ์" ขึ้นมาในหน้าจอแล้ว

ทำไมถึงงงแบบนี้ได้ — เปรียบเทียบง่ายๆ

Terminal (หน้าต่างสีดำที่พิมพ์คำสั่ง) ทำงานแบบ "คิวเดียว ทำทีละอย่าง" เหมือนพนักงานเคาน์เตอร์ที่รับออเดอร์ได้ทีละคน — ถ้าลูกค้าคนแรก (คำสั่งทดสอบ 5 step) ยังไม่จ่ายเงินเสร็จ (ยังรันไม่จบ) ต่อให้ลูกค้าคนที่สอง (คำสั่งเทรนเต็ม) เดินเข้ามาพูดออเดอร์ไป แล้ว พนักงานก็ยังไม่เริ่มทำ — แค่ว่า "จำค่าพูดไว้ก่อน" รอคิวเต็มจบก่อนจริงๆ ถึงจะเริ่มทำออเดอร์ใหม่

สิ่งที่เกิดขึ้นจริง: คำสั่งทดสอบ 5 step แม้จะ "เทรนเสร็จ" และ "เซฟไฟล์เสร็จ" ไปแล้ว แต่ตัวสคริปต์ (`train_qwen36.py`) ยังมีขั้นตอนท้ายสุดที่ยังไม่เสร็จอยู่ — คือการให้โมเดลลองตอบตัวอย่างจริง 3 ข้อ (เพื่อโชว์ผลลัพธ์ทันทีให้ดู ไม่ใช่แค่ดูตัวเลข loss เฉยๆ) ขั้นตอนนี้ใช้เวลานาน (โมเดลต้อง "คิดคำตอบ" ทีละตัวอักษรจริงๆ) เพราะฉะนั้นโปรแกรมเดิมยังไม่จบจริง แม้จะดูเหมือนจบไปแล้วก็ตาม

วิธีตรวจสอบ (สืบสวนแบบนักสืบ)

ใช้คำสั่งเช็ค "**process**" (โปรแกรมที่กำลังทำงานอยู่ในเครื่อง) พบว่ายังมีโปรแกรมตัวเดิมทำงานอยู่ (รันมาแล้ว 885 วินาที = เกือบ 15 นาที ซึ่งนานกว่าที่ทดสอบ 5 step ควรใช้เวลา) แล้วเช็คความมันเขียน log ไปที่ไฟล์ไหน พบว่ายังเขียนไปที่ไฟล์เดิม (`test_run.log`)

ไม่ใช่ไฟล์ใหม่ (`full_train.log`) — ยืนยันชัดเจนว่าคำสั่งใหม่ที่พิมพ์ไปยังไม่ได้เริ่มทำงานจริง แค่ออก "จำไว้รอคิว" อยู่

บทเรียน

เวลาทำงานผ่าน terminal แบบนี้ ต้องรอให้เห็น **prompt** (เครื่องหมายพร้อมพิมพ์คำสั่งใหม่) กลับมาจริงๆ ก่อน ถึงจะพิมพ์คำสั่งต่อไปได้ ถ้าพิมพ์แซงคิวไปก่อน คำสั่งจะไม่หายไปไหน (ไม่ต้องพิมพ์ซ้ำ) แต่ต้องรอให้คิวเดิมเคลียร์ก่อนมันถึงจะเริ่มทำงานเองอัตโนมัติ — วิธีเช็คที่แม่นยำกว่าการดูหน้าจอเฉยๆ คือเช็ค "process" ว่างงานเดิมทำงานอยู่จริงไหม เหมือนที่ทำในเหตุการณ์นี้

13. เหตุการณ์ "เทรนเสร็จแล้วดันพัง" — บทเรียนเรื่อง "environment ใช้ร่วมกัน"

เทรนเต็ม 3 epoch จบสำเร็จ (loss ลดจาก 2.0 เหลือ 0.14 — ตีมาก) **เซฟไฟล์ผลลัพธ์ (LoRA adapter)** สำเร็จแล้วด้วย แต่ทันทีหลังจากนั้น ขั้นตอนโบนัšťท้ายสคริปต์ (ลองให้โมเดลตอบตัวอย่าง 3 ข้อโห้ผล) ดันพังด้วย error หาไฟล์ไม่เจอ

สาเหตุ — ความผิดพลาดของ Claude เอง

ระหว่างที่รอเทรนอยู่ Claude เปิดหน้าต่าง terminal อีกอันแยกไว้ เพื่อเตรียมโปรแกรม `llama.cpp` ล่วงหน้า (สำหรับขั้นตอนแปลงโมเดลเป็น GGUF ที่คุยกันไว้) แล้วสั่ง `pip install` (โปรแกรมติดตั้งซอฟต์แวร์ของ Python) ในหน้าต่างนั้น

ปัญหาคือ: ทั้ง 2 หน้าต่าง (เทรนโมเดล + เตรียม llama.cpp) ใช้ "ห้องเก็บของ" เดียวกัน (Python environment เดียวกันทั้งเครื่อง) — พอ `pip install` ของ llama.cpp ไปติดตั้ง/อัปเดตไลบรารีชื่อ `transformers` (ไลบรารีหลักที่ใช้รันโมเดล AI) มันดันไปเขียนทับเวอร์ชันเดิมที่ Unsloth เตรียมไว้เฉพาะสำหรับโมเดลนี้ (ที่มีไฟล์รองรับ Qwen3.5 MoE อยู่) กลายเป็นเวอร์ชันมาตรฐานทั่วไปที่ไม่มีไฟล์นั้น — พอสคริปต์เทรน (ที่ยังรันค้างอยู่ ยังไม่จบ) ต้องกลับไปใช้ไฟล์นั้นอีกครั้งหลังเซฟเสร็จ (ตอนจะลองโห้ตัวอย่าง 3 ข้อ) มันเลยหาไม่เจอแล้วพัง

เปรียบเทียบง่ายๆ

เหมือน 2 คนใช้ "ตู้เครื่องมือ" ใบเดียวกันพร้อมกัน — คนหนึ่ง (งานเทรน) กำลังใช้ไขควงชิ้นหนึ่งอยู่ อีกคน (งานเตรียม llama.cpp) ดันเอาไขควงชิ้นเดียวกันไปเปลี่ยนเป็นรุ่นอื่นระหว่างที่อีกฝ่ายยังใช้อยู่ พอคนแรกจะหยิบไขควงมาใช้ต่อ (ชิ้นเดิมที่คุ้นเคย) มันกลายเป็นคนละชิ้นไปแล้ว

บทเรียนสำคัญ

ห้ามติดตั้ง/แก้ไขซอฟต์แวร์ที่ใช้ร่วมกัน (pip install) ขณะมีงานสำคัญอื่นกำลังรันอยู่ในเครื่องเดียวกัน ถ้าจะทำงานเสริมระหว่างรอ ต้องแยก "ห้องเก็บของ" ให้ชัดเจน (เทคนิคที่เรียกว่า **virtual environment**) ไม่ใช่ห้องเดียวกับงานหลักที่กำลังรันอยู่ — โชคดีที่รอบนี้ไฟล์ผลลัพธ์สำคัญ (LoRA adapter) เซฟเสร็จไปก่อนพังพอดี ไม่จันอาจต้องเทรนใหม่ทั้งหมด (เสียเวลา+เงินไปฟรีๆ)

ผลกระทบ: ตอนนี้ต้องซ่อม environment กลับให้ถูกต้องก่อน ถึงจะวัดผลคุณภาพโมเดล (`eval_fields.py --adapter`) ได้ตามแผน

14. ผลวัดคุณภาพจริง — การทุนได้ผลใหม่ (คำตอบ: ได้ผลชัดเจน)

หลังซ่อม environment เสร็จ วัดผลโมเดลที่ทุนแล้วเทียบกับ "ก่อนทุน" (baseline ที่วัดไว้ตอนแรก):

วัดอะไร	ก่อนทุน	หลังทุน
ตอบเป็น JSON ที่ใช้งานได้	0%	90%
โครงสร้างภาพรวม (view) ตรง	0%	60%
หารายการหลัก/คาน/เสาเจอ	0%	9.4%

แปลว่า: ก่อนทุน โมเดลตอบมั่วจนแม้แต่จะอ่านเป็นข้อมูลก็ยังไม่ได้เลย (0%) หลังทุน ตอบถูกรูปแบบเกือบทุกครั้ง (90%) — **นี่คือหลักฐานว่าการทุนได้ผลจริง** ส่วนที่ยังต้องปรับปรุง (รอบทุนหน้า) คือมันยัง "มองข้าม/เกิน" รายการหลักบางส่วน (9.4% หาเจอ) — เป็นโจทย์สำหรับรอบทุนครั้งถัดไป

15. ทำไม "แปลงเป็น GGUF" ถึงมีหลายขั้นตอน + เจอปัญหาอะไรบ้าง

พยายามแปลงโมเดลที่ทุนแล้วให้เป็นไฟล์เดียวที่รันบนคอมพิวเตอร์ได้ (GGUF) ทำสำเร็จแต่เจอปัญหาระหว่างทางหลายจุด (อธิบายรวบให้เข้าใจง่าย):

- "รวม" (merge) โพลิตอิท (LoRA จากข้อ 6) เข้ากับโมเดลต้นฉบับให้เป็นก้อนเดียว — ทำสำเร็จ
- แปลงเป็น GGUF ลองแบบเล็ก (q8_0) ก่อนเพื่อประหยัดพื้นที่ แต่เจอว่าโปรแกรมบีบอัด "ไม่ยอมบีบซ้ำ" ไฟล์ที่บีบไปแล้ว (เหมือนบีบน้ำผลไม้ที่คั้นไปแล้วซ้ำไม่ได้) ต้องแปลงใหม่จากไฟล์ความละเอียดเต็ม (f16, ใหญ่กว่า ~70GB)
- ดิสก์เต็มหลายรอบ — เครื่องเช่ามีพื้นที่จำกัด (150GB) ต้องคอยลบไฟล์เก่าที่ไม่ใช้แล้วตลอดทาง (checkpoint เก่า, โมเดลต้นฉบับที่แคชไว้หลังโหลดเข้า RAM แล้ว, ไฟล์กลางระหว่างแปลง) เหมือนจัดกระเป๋าเดินทางที่ต้องเอาของเก่าออกก่อนใส่ของใหม่เข้า
- บีบสำเร็จ: ได้ไฟล์ `Q4_K_M` ขนาด 21.2GB — รันบนคอมพิวเตอร์ (RAM 31GB + VRAM 8GB) ได้จริง

⚠️ ปัญหาที่ยังไม่แก้: โมเดลนี้ต้อง "อ่านภาพ" ได้ด้วย (ไม่ใช่แค่ข้อความ) แต่ส่วนอ่านภาพ (เรียกว่า mmproj) ยังแยกออกมาเป็น GGUF ไม่สำเร็จ เพราะโมเดลนี้ใหม่มาก เครื่องมือ (`llama.cpp`) เขียนเตือนเองว่า "ใช้ได้แค่บางโมเดล" — ต้องลองต่อวันหลัง ตอนนีไฟล์ GGUF ที่ได้ใช้ได้แค่กับงานที่เป็นข้อความล้วน

16. บทเรียนสุดท้ายของคืนนี้ — ทำลาย instance ก่อนสำรองไฟล์ออกมา

มะขามปิด instance ที่เช่าไว้เอง ก่อน ที่จะดาวน์โหลด/อัปโหลดไฟล์ผลลัพธ์ (LoRA adapter, โมเดล merge, ไฟล์ GGUF) ออกมาเก็บที่อื่น — เครื่องเช่าแบบนี้เก็บไฟล์แบบ "ชั่วคราว" พอปิด/ทำลายเครื่อง ไฟล์ข้างในหายหมดทันที ไม่มีทางกู้คืน

เปรียบเหมือนเช่าโรงแรมทำงาน พอเช็คเอาท์แล้วข้าวของที่ลืมไว้ในห้องหายหมด — ต้องขนของออกมาเก็บที่บ้าน (หรือฝากที่ปลอดภัยอื่น เช่น HuggingFace) ก่อน เช็คเอาท์เสมอ

ผลที่เกิดขึ้นจริง: งานเทรน+แปลง GGUF ของทั้งวันนี้ต้องทำใหม่หมดถ้าจะเอาโมเดลจริงมาใช้ — แต่ไม่เสียเวลาทั้งหมดเปล่า เพราะสคริปต์ที่แก่นักครบแล้ว + บทเรียนที่บันทึกไว้ในไฟล์นี้และ diary ยังอยู่ครบ รอบหน้าจะรันได้เร็วและราบรื่นกว่านี้มาก ไม่ต้องเจอบั๊กเดิมซ้ำอีกเลย

ตอนที่ 2 (2026-07-24) — วันนี้ทำอะไร "ก่อน" เข้าการดจ่อ ทำไมต้องทำขนาดนี้

ไฟล์นี้ต่อเนื่องจากตอนที่ 1 ด้านบน (ศัพท์พื้นฐานอย่าง VRAM, LoRA, MoE, OOM อธิบายไว้ที่นั่นแล้ว จะไม่พูดซ้ำที่นี่) วันนี้ยังไม่ได้เข้าการดจ่อเลย — ทั้งหมดที่ทำวันนี้คือ "เช็คให้แน่ใจก่อน" ตามที่ตกลงกันไว้หลังเหตุการณ์วันที่ 21

1. ทำไมวันนี้ต้องเสียเวลาเช็คตั้งเยอะ ทั้งที่ยังไม่ได้ทำอะไรเลย

เหตุการณ์วันที่ 21 (ดูตอนที่ 1 ข้อ 16) ไม่ได้พังเพราะบั๊กโค้ด — พังเพราะ "เดือนไม่ทันเวลา" (บอกว่าไฟล์ยังไม่ backup แบบเบาๆ ไม่ใช่ค่าเดือนหนักแน่น) กับ "บอกว่าพร้อมใช้ทั้งที่มันยังอ่านภาพไม่ได้"

บทเรียนที่ได้คือ: ทุกอย่างที่เราเช็คได้ฟรีบนคอมพิวเตอร์ตัวเอง ต้องเช็คให้จบก่อนควักเงินเข้าการดจ่อ ไม่ใช่ไปเจอปัญหาแล้วมานั่งแก้ตอนนาฬิกากำลังเดินคิดเงินอยู่ (ตอนนั้น 30 บาท/ชั่วโมง) — นี่คือที่มาของ "Phase 0" ใน `RETUNE_WORKFLOW.md` (ปัจจุบันคือ `t01_workflow.md` — ดูภาคผนวก A) และกลายเป็น Rule 4 ใหม่ ใน `rule_of_tune.md` วันนี้: ก่อนเข้าจริงทุกครั้ง ต้องรันเช็ค Phase 0 จริง ไม่ใช่แค่มีเอกสารเช็คลิสต์ไว้เฉยๆ

2. 0.1 — ทดสอบกุญแจ HuggingFace ก่อนใช้จริง

HuggingFace (จากข้อ 4 ในตอนที่ 1) คือที่เก็บโมเดล — วันนี้เพิ่มบทบาทใหม่: เป็นที่ที่เราจะ อัปโหลดผลลัพธ์การทวนไปเก็บสำรอง (กันเหตุการณ์แบบวันที่ 21 ซ้ำ)

ในการอัปโหลด ต้องมี "กุญแจ" (token) - เหมือนรหัสผ่านพิเศษที่พิสูจน์ว่าเราเป็นเจ้าของบัญชีจริง วันนี้ทดสอบกุญแจนี้ด้วยการยิงคำถามง่ายๆ ไปถาม HuggingFace ว่า "กุญแจนี้เป็นใคร" — ได้คำตอบกลับมาถูกต้อง (ชื่อบัญชี, สิทธิ์เขียนไฟล์) แปลว่ากุญแจใช้ได้จริง เช็คตอนนี้ฟรี ดีกว่าไปเจอว่ากุญแจผิดตอนเข้าการดจ่อไปแล้ว

3. 0.2 — เช็คเงินในบัญชี Vast.ai + จุดที่เกือบพลาดอีกรอบ

เช็คยอดเงินจริงในบัญชี Vast.ai (มะขามส่งภาพหน้าจอมามาให้ดู) — มี \$12.29 เพียงพอกับงบที่ประเมินไว้ (\$5-10)

แต่เจอจุดสำคัญจากภาพหน้าจอนั้นเอง: หน้าเว็บ Vast.ai ตั้งค่า "ขนาดพื้นที่เก็บข้อมูล" (Container Size / Disk) เริ่มต้นไว้ที่ 150GB — แต่จากบทเรียนวันที่ 21 (ดิสก์เต็มหลายรอบ, ข้อ 15 ในตอนที่ 1) รู้แล้วว่ารอบนี้ต้องการอย่างน้อย 300GB ถ้าไม่ได้สังเกตแล้วกดเข้าไปตามค่าเริ่มต้น จะเจอปัญหาดิสก์เต็มซ้ำแบบเดิมทันที — นี่คือนิสัยที่จริงว่าทำไม Phase 0 ถึงคุ้มที่จะเสียเวลาเช็ค เพราะจุดนี้ถ้าไม่เช็คก่อน จะไปเจอลางทางตอนเสียเงินแล้ว

4. 0.3 — "ซ้อมของจริง" บนคอมพิวเตอร์ตัวเองก่อน (local dry run)

นี่คือขั้นที่ป้องกันปัญหาวันที่ 21 ได้ตรงจุดที่สุด อธิบายทีละค่าใหม่:

GGUF คืออะไร

โมเดล AI ปกติ (ตอนเทรน) เก็บในรูปแบบที่กินพื้นที่มากและรันได้เฉพาะบนการ์ดจอแรงๆ GGUF คือ "รูปแบบไฟล์" ที่บีบอัดโมเดลให้เล็กลงและรันได้บนคอมทั่วไป (แม้จะไม่มีการ์ดจอแรงมาก) — เป้าหมายสุดท้ายของทั้งโปรเจกต์นี้คือได้ไฟล์ GGUF มาไว้ใช้ในคอมมะขามเอง

Quantization (การบีบอัด) คืออะไร

โมเดลเก็บตัวเลขแบบละเอียดมาก (เช่น bf16 = ตัวเลข 16 บิต) Quantization คือการบีบตัวเลขให้หยาบขึ้น (เช่น 4 บิต) เพื่อให้ไฟล์เล็กลงมาก แลกกับความแม่นยำที่ลดลงเล็กน้อย — เหมือนบีบอัดรูปภาพ JPEG คุณภาพสูง เป็น JPEG คุณภาพต่ำ ไฟล์เล็กลงเยอะ ภาพยังดูออกแต่รายละเอียดหายไปนิดหน่อย

mmproj คืออะไร — ทำไมเป็นปัญหาใหญ่วันที่ 21

โมเดลที่ใช้ต้อง "อ่านภาพได้" ไม่ใช่แค่อ่านตัวหนังสือ — ส่วนที่ทำหน้าที่ "แปลงภาพเป็นตัวเลขให้โมเดลเข้าใจ" แยกเป็นไฟล์ต่างหาก เรียกว่า **mmproj** วันที่ 21 ไฟล์นี้แยกออกมาไม่สำเร็จ (โมเดลใหม่เกินไป เครื่องมือไม่รองรับ) ทำให้ได้ GGUF ที่อ่านได้แค่ตัวหนังสือ อ่านภาพแบบก่อสร้างไม่ได้เลย — **ทั้งที่นั่นคืองานหลักของโปรเจกต์นี้**

ข่าวดีที่ค้นพบหลังจากนั้น: เพราะสคริปต์เทรน (`train_qwen36.py`) ตั้งค่าไว้ว่า "ห้ามแก้ส่วนอ่านภาพ" (`finetune_vision_layers=False` — สอนแค่ภาษา ไม่แตะส่วนอ่านภาพเลย) แปลว่าส่วนอ่านภาพของโมเดลที่ทุนแล้ว เหมือนเดิมกับโมเดลต้นฉบับ **100%** — เลยไม่ต้องพยายาม "แยกไฟล์ใหม่" อีกต่อไป แคไปโหลดไฟล์ mmproj ที่บริษัทเจ้าของโมเดลทำไว้แล้ว (899MB) มาใช้คู่กับ GGUF ที่ทุนเองได้เลย — ปัญหาที่เคยติดค้างวันที่ 21 แก้ได้แบบไม่ต้องง้อเครื่องมือที่ไม่รองรับอีกต่อไป

llama.cpp คืออะไร

llama.cpp คือโปรแกรมที่ใช้ "รัน" ไฟล์ GGUF บนคอมพิวเตอร์ทั่วไป — ต้องมีทั้งไฟล์โมเดล (GGUF) และไฟล์ mmproj ป้อนเข้าไปพร้อมกัน ถ้ายากให้อ่านภาพได้

สิ่งที่ทำจริงวันนี้ (ซ่อมทั้งกระบวนการ ก่อนเสียเงินเช่า)

1. โหลดไฟล์ mmproj ตัวจริง (899MB) — ของจริงที่จะใช้ตอนทุนเสร็จ
2. โหลดโมเดลตัวอย่างเล็กๆ (ยังไม่ทุน ขนาดบีบอัด ~10.8GB) มาแค่ "ทดสอบว่าระบบรันได้" ไม่ได้สนใจว่าตอบเก่งไหม
3. โหลดโปรแกรม llama.cpp มาติดตั้ง
4. ป้อนภาพแบบก่อสร้างจริง (บ้าน01 หน้า19) เข้าไปถามว่า "อธิบายภาพนี้หน่อย"
5. **ผลลัพธ์: ผ่าน** — โมเดลอ่านตัวหนังสือไทยในภาพถูก ("แผนลunฐานรากแผ่ และฐานรากเสาเข็ม"), จับ grid ถูก (1/2/3 x A/B/C/D), อ่าน label ฐานรากถูก (F1,C1 ฯลฯ)

นี่พิสูจน์อะไร: คอมมะขามเอง (RTX 5060, VRAM แค 8GB) **รันโมเดลแบบนี้ได้จริง** (แม้ VRAM ไม่พอ ต้องแบ่งงานไป RAM ปกติบางส่วน) และ mmproj ใช้งานได้จริง — พอทุนเสร็จจริง จะแค่ "เปลี่ยนไฟล์โมเดล" ไฟล์เดียว ไม่มีอะไรน่าแปลกใจเพิ่มอีกแล้ว ต่างจากวันที่ 21 ที่เพิ่งมารู้อาว่า mmproj มีปัญหาตอนงานเกือบจบ

ทดสอบเสร็จแล้วลบไฟล์โมเดลทดสอบ (10GB) ทั้ง เก็บแค่ mmproj ไว้ใช้ต่อ

5.0.4 — อ่านสคริปต์ซ้ำจริงจัง เจอบักที่ไม่เคยรู้มาก่อน

Token คืออะไร (ศัพท์ใหม่วันนี้)

โมเดล AI ไม่ได้อ่านตัวอักษรทีละตัวหรือคำทีละคำแบบคนอ่าน — มันตัดข้อความ (และภาพ) เป็นชิ้นเล็กๆ เรียกว่า **token** แล้วประมวลผลทีละชิ้น ยิ่งข้อความ/ภาพยาว/ละเอียดเท่าไร ก็ยังมี token เยอะขึ้นเท่านั้น

ภาพ 1 ในก๊นับเป็น token ได้เหมือนกัน — ภาพทีละเอียดกว่าจะถูกตัดเป็น token เยอะกว่า (สูตรของโมเดลนี้: ภาพที่ใหญ่ที่สุดที่กำหนดไว้ = ประมาณ 5,120 token/ภาพ)

MAX_LENGTH คืออะไร — ทำไมเป็นบัก

สคริปต์เทรนต้องกำหนด "ความยาวสูงสุดที่ยอมรับได้ต่อ 1 ตัวอย่าง" (`MAX_LENGTH`) ไว้ล่วงหน้า — ถ้าตัวอย่างไหนยาวเกินที่กำหนด โปรแกรมจะ "ตัดท้ายทิ้ง" ส่วนที่เกินมา

ปัญหาที่เจอวันนี้: ค่าที่ตั้งไว้เดิม (9,216 token) **คิดจากตัวอย่างที่มีภาพแค่ 1 ใบเท่านั้น** แต่ในชุดข้อมูลจริงมีตัวอย่างพิเศษ 5 ตัว (เรียกว่า "grid master" — 1 ตัวต่อบ้าน สอนเรื่องเส้น grid ทั้งหมด) ที่ใช้ภาพพร้อมกัน **2-4 ใบ** — พอนับจริงๆ ตัวอย่างพวกนี้ต้องการพื้นที่ถึง ~22,000 token (มากกว่าที่ตั้งไว้ถึง 2.4 เท่า)

แปลว่าอะไร: ถ้าไม่แก้ ตอนเทรนจริง โปรแกรมจะตัดท้ายตัวอย่างพวกนี้ทิ้ง — และท้ายตัวอย่างคือคำตอบที่ถูกต้องที่ต้องการสอนโมเดล (ไม่ใช่แค่ตัดส่วนที่ไม่จำเป็น) เท่ากับสอนโมเดลด้วยคำตอบที่ขาดๆ หายๆ พอตีตัวอย่างพวกนี้คือชุดข้อมูลเดียวที่สอนเรื่อง "เส้น grid ทั้งหมด" (จุดที่บอกไว้ในสคริปต์เองว่าเป็นส่วนที่พลาดบ่อยที่สุด) — ถ้าปล่อยไว้ การทุนรอบนี้จะสอนเรื่องนี้ผิดตั้งแต่ต้น

ทางเลือกที่มีให้เลือก + สิ่งที่มาเลือก

มี 4 ทางแก้ (เพิ่มขนาดที่ยอมรับได้ / ลดความละเอียดภาพเฉพาะกลุ่มนี้ / ปลดปล่อยไว้ตามเดิม / ตัดตัวอย่างกลุ่มนี้ทิ้งไปเลย) — มาเลือกเพิ่มขนาดที่ยอมรับได้ (จาก 9,216 เป็น 24,576) เพื่อไม่ให้เสียรายละเอียดของภาพเลย

ข้อแลกเปลี่ยนที่ต้องรู้: ค่าที่สูงขึ้นนี้ทำให้ตอนเรน 5 ตัวอย่างนี้โดยเฉพาะ มีโอกาสหน่วยความจำเต็ม (OOM, ดูข้อ 8 ในตอนที่ 1) มากขึ้น เพราะรอบก่อนหน่วยความจำเหลือแค่ ~100MB จาก 95GB ตอนตั้งไว้ที่ 9,216 — ถ้าเจอ OOM ตรงจุดนี้ วิธีแก้คือลดขนาด "โพสต์อิท LoRA" ต่อก็ค หรือลดความละเอียดภาพเฉพาะ 5 ตัวอย่างนี้ ไม่ใช่ลดค่านี้กลับไปทีเดียว (เพราะจะกลับไปตัดคำตอบทั้งอีกรอบ)

จุดสำคัญ: บั๊กนี้มีอยู่ตั้งแต่ก่อนเพิ่มบ้าน 05 เข้าไปด้วยซ้ำ — ไม่มีใครเคยตรวจพบมาก่อนเพราะไม่เคยอ่านสคริปต์เทียบกับตัวเลขจริงในชุดข้อมูลอย่างละเอียดขนาดนี้ นี่คือเหตุผลที่ Rule 4 (ข้อ 1) เขียนไว้ว่าต้อง "ทำและเช็จริง ไม่ใช่แค่อ่านผ่านๆ"

6.0.5 — เลือกเครื่องเช่าจริงจากรายการที่มีอยู่จริง

จากภาพหน้าจอ Vast.ai ที่มาส่งมา เลือก m:67820 — การ์ด RTX PRO 6000 (VRAM 96GB, พอสำหรับโมเดลที่ต้องการอย่างน้อย 74GB), ราคา \$0.975/ชั่วโมง, reliability 99.76%

Reliability (ความน่าเชื่อถือ) คืออะไร: เปอร์เซนต์ที่บอกว่าเครื่องเช่าเครื่องนี้เคยทำงานล้ม/หลุดกลางทางบ่อยแค่ไหนในอดีต (คำนวณจากประวัติการใช้งานจริงของคนเช่าคนอื่น) ยิ่งใกล้ 100% ยิ่งเชื่อถือได้ — เลือกเครื่องที่ reliability ต่ำเสี่ยงเครื่องหลุดกลางทางตอนกำลังเรนอยู่ (เสียเวลา+เงินฟรี)

6.5. llama.cpp คืออะไรกันแน่ — ทำไมไม่ใช่ Unsloth รันตอนใช้งานจริง

ข้อ 4 พูดถึง llama.cpp สั้นๆ ไปแล้ว (โปรแกรมที่รัน GGUF) แต่มีคำถามดีที่โผล่มาที่หลังตอนใกล้จะถึง Phase 10 (ทดสอบรันบนคอมพิวเตอร์ตัวเอง): "ตอน run ขอใช้ Unsloth แทนได้ไหม" — คำตอบคือไม่เหมาะ และเหตุผลช่วยให้เข้าใจภาพรวมทั้งหมดชัดขึ้น

Unsloth กับ llama.cpp ทำหน้าที่คนละอย่างกัน คนละช่วงเวลากัน:

	Unsloth	llama.cpp
ใช้ตอนไหน	ตอนเรน (บนเครื่องเช่า GPU แรงๆ)	ตอนใช้งานจริง (บนคอมพิวเตอร์ตัวเอง)
ต้องการอะไร	โมเดลเต็ม bf16 (~70GB) + การ์ดจอ VRAM 74GB+	ไฟล์ GGUF ที่บีบแล้ว (~21GB) รันได้แม้ VRAM แค่ 8GB
เปรียบเทียบ	เหมือน "โรงงานผลิต" ที่ต้องใช้เครื่องจักรใหญ่	เหมือน "เครื่องเล่น" ที่ใครก็เปิดดูของที่ผลิตเสร็จแล้วได้

ถ้าลองเอา Unsloth มารันบนคอมพิวเตอร์ (VRAM 8GB) จะรับไม่ไหวเลย เพราะ Unsloth ยังคาดหวังโมเดลขนาดเต็มแบบเดียวกับตอนเรน — นี่คือเหตุผลทั้งหมดที่ต้องเสียเวลาทำ GGUF + quantize (ข้อ 4 ด้านบน): บีบโมเดลให้เล็กพอที่ llama.cpp จะรันบนเครื่องเล็กๆ ได้ ถ้าข้ามขั้นตอนนี้ไปเลย จะไม่มีทาง "รันบนคอมพิวเตอร์ตัวเอง" ได้จริงตามเป้าหมายที่ตั้งไว้แต่แรก (ดูเป้าหมายในข้อ 1 ของตอนที่ 1)

พูดง่าย: Unsloth = เครื่องมือช่างที่ใช้ตอนสร้าง, llama.cpp = รีโมทที่ใช้ตอนดู สองอย่างนี้ไม่แทนกันได้เลย เพราะออกแบบมาคนละงาน

7. สรุป: ทำครบ Phase 0 แล้ว พร้อมเช่าจริงหรือยัง

ขั้น	ผล
0.1 ทุนแฉ HuggingFace	✅ ทดสอบผ่านจริง
0.2 เงินในบัญชี Vast.ai	✅ \$12.29 พอ + เจอจุด disk 150GB ต้องแก้เป็น 300
0.3 ช้อมูลรันของจริงบนคอมตัวเอง	✅ อ่านภาพแบบก่อสร้างได้จริง
0.4 อ่านสคริปต์ซ้ำ	✅ เจอบั๊ก MAX_LENGTH จริง แก้แล้ว
0.5 เลือกเครื่องเช่า	✅ เลือก m:67820 ไว้แล้ว

พร้อมเช่าจริงได้แล้ว — สิ่งเดียวที่ต้องจำตอนกด Rent จริง: ลากตัวเลือกพื้นที่เก็บข้อมูล (disk) จาก 150 เป็น 300 ก่อนกดยืนยัน (ข้อ 3 ด้านบน)

8. คำศัพท์ใหม่วันนี้ (ต่อจากท้ายเล่มในตอนที่ 1)

คำศัพท์	ความหมายสั้นๆ
GGUF	รูปแบบไฟล์โมเดลที่บีบให้เล็กลง รันบนคอมทั่วไปได้
Quantization	การบีบตัวเลขในโมเดลให้หยาบขึ้น ไฟล์เล็กลง แลกความแม่นยำนิดหน่อย
mmpromj	ไฟล์ส่วน "แปลงภาพเป็นตัวเลข" ของโมเดล แยกจากไฟล์โมเดลหลัก
llama.cpp	โปรแกรมที่ใช้รันไฟล์ GGUF บนคอมทั่วไป (คนละตัวกับ Unsloth ที่ใช้ตอนเทรน — ดูข้อ 6.5)
Token	หน่วยย่อยที่โมเดลตัดข้อความ/ภาพออกมาประมวลผลทีละชิ้น
MAX_LENGTH / sequence length	ความยาวสูงสุด (นับเป็น token) ที่ 1 ตัวอย่างเทรนใช้ได้ ยาวเกินจะถูกตัดทิ้ง
Truncation (การตัดทิ้ง)	เมื่อตัวอย่างยาวเกิน MAX_LENGTH ส่วนที่เกินจะถูกตัดออกจริง ไม่ใช่แค่เตือน
Reliability (%)	ความน่าเชื่อถือของเครื่องเช่า วัดจากประวัติเครื่องล่ม/หลุดในอดีต
Dry run (ซ้อมของจริง)	การทดสอบกระบวนการทั้งหมดด้วยของจำลอง/ของเล็ก ก่อนใช้ของจริง/เสียเงินจริง

ตอนที่ 3 (2026-07-28) — วันนี้ t02 ลงมือเข้าจริง เกิดอะไรขึ้นบ้าง

ไฟล์นี้ต่อจากตอนที่ 2 ด้านบน (ศัพท์พื้นฐานอย่าง GGUF, mmproj, MAX_LENGTH, VRAM, LoRA, Reliability อธิบายไว้ที่นั่นแล้ว ไม่พูดซ้ำที่นี่) วันที่ 24 คือ "ซ้อมเช็คก่อนเข้า" (Phase 0) ส่วนวันนี้คือวันที่เข้าการดจจจริงและลงมือทุนจริงของรอบ t02 — ตั้งแต่กด Rent ยันวัดผล เจอทั้งบั๊กใหม่ ทั้งผลลัพธ์ที่ไม่ได้เป็นไปตามหวัง

หมายเหตุ: ตอนนี้นำถึง t02 (โมเดลตระกูล Qwen3-VL) ไม่ใช่ t01 — เก็บไว้ในตอนที่ 3 นี้เพราะเป็นการทดลองเปรียบเทียบกับ t01 โดยตรง และบทเรียนที่ได้ (โดยเฉพาะข้อ 9) ย้อนกลับไปกระทบความเชื่อมั่นเกี่ยวกับ t01 เองด้วย

1. ทบทวน: t02 ตอบคำถามอะไร

t01 (ทำไปแล้ววันที่ 21-24) ทุนโมเดล Qwen3.6-35B-A3B ได้ผลดี (JSON valid 90%, element recall 28.2%) แต่ผมขามอยากรู้ว่า โมเดลอีกตระกูลหนึ่งชื่อ Qwen3-VL จะทำได้ดีกว่าไหม ถ้าจะเทียบให้แฟร์ ต้องคุมทุกอย่างให้เหมือนกันหมด (dataset เดียวกัน, การตั้งค่าเดียวกัน) เปลี่ยนแค่ตัวโมเดล — นี่คือนี่ที่เรียกว่า "การทดลองแบบมีตัวควบคุม" (controlled experiment) เหมือนทดลองวิทยาศาสตร์ที่เปลี่ยนตัวแปรทีละตัว ถ้าเปลี่ยนหลายอย่างพร้อมกัน จะไม่รู้ว่าผลต่างมาจากอะไรกันแน่

2. เจอบั๊กใหม่ที่ไม่เคยเจอในรอบ t01 — เพราะ "เวอร์ชันไลบรารีขยับ"

ปัญหา

พอเริ่มรันจริง (Phase 4) โปรแกรม error ทันที:

```
AttributeError: property of 'Qwen2VLImageProcessor' object has no setter
```

อธิบายง่ายๆ ว่าเกิดอะไรขึ้น

โค้ดที่เขียนไว้ตั้งค่า "ความละเอียดภาพ" ด้วยการเขียนทับ `ip.max_pixels = ...` ตรงๆ — ตอนเขียน โค้ดไว้ (ก่อนหน้านี้) วิธีนี้ใช้ได้ แต่พอเข้าเครื่องใหม่วันนี้ ตัวไลบรารี `transformers` ที่ pip ติดตั้งให้กลายเป็นเวอร์ชันใหม่กว่าที่เคยทดสอบ (5.5.0) ผู้พัฒนาไลบรารีเปลี่ยนวิธีตั้งค่าไปแล้ว ทำให้ทางเก่าใช้ไม่ได้อีกต่อไป

เปรียบเทียบ: เหมือนแอปในมือถือที่อัปเดตตัวเองอัตโนมัติ แล้ววันหนึ่งปุ่มที่เคยกดได้ก็หายไป เพราะบริษัทเปลี่ยนหน้าตาแอป — โค้ดของเราก็เจอแบบเดียวกัน แค่เป็นไลบรารีแทนแอป

ทำไมไม่มีใครรู้ล่วงหน้า

โค้ดตัวนี้ (`train_qwen3vl.py`) เขียนไว้ตั้งแต่ก่อนที่ dataset จะโด่งขึ้น แล้วไม่เคยถูกรันจริง มาก่อนเลย จนกระทั่งวันนี้ — ไลบรารีขยับเวอร์ชันไปเรื่อยๆ ระหว่างที่โคดนอนอยู่เฉยๆ บทเรียน: โค้ดที่เขียนไว้นานแล้วไม่เคยรัน มีความเสี่ยงสูงกว่าที่คิด เพราะโลกรอบๆ มันเปลี่ยนไป ตลอด (ไลบรารีอัปเดต) ทั้งที่ตัวโคดเองไม่ได้มีคนเขียน

วิธีแก้จริง

เปิดดู source code จริงของไลบรารี (ไม่ใช่เดา) เจอว่าทางที่ยังใช้ได้จริงคือ `ip.size["longest_edge"] = ...` — แก้ให้ใช้ทางนี้ทางเดียว ลบทางเก่าทิ้ง ทดสอบซ้ำผ่าน

3. อ่านกราฟ loss ตอนเทรน — loss คืออะไร บอกอะไรเรา

loss คือตัวเลขที่บอกว่า "โมเดลหายผิดไปมากแค่ไหน" ในแต่ละรอบ — ยิ่งต่ำยิ่งดี (0 = หายถูก เป๊ะ, ตัวเลขสูง = หายมั่วมาก) วันนี้เห็นค่าจริงตลอดการเทรน:

เริ่มต้น: 1.795 → ลงเรื่อยๆ → จบที่ประมาณ 0.18-0.21

เส้นทางลงแบบนี้คือสัญญาณที่ดี — แปลว่าโมเดลกำลังเรียนรู้จริง ไม่ใช่แค่อยู่กับที่หรือแฉ่ง (ถ้า loss ไม่ลงเลยหรือขึ้นๆ ลงๆ มั่วไม่มีทิศทาง = สัญญาณว่ามีอะไรผิดปกติ เช่น learning rate สูงเกินไป)

แต่ loss ต่ำ ≠ ใช้งานได้จริง — นี่คือจุดสำคัญที่ทั้งไฟล์ `rule_of_tune.md` เตือนไว้ (บทเรียน ข้อ 4 ของไฟล์ CLAUDE.md เดิม) loss วัดแค่ "ทายใกล้คำตอบแค่ไหนในทางคณิตศาสตร์" ไม่ได้วัดว่า อ่านเลขหลักถูกจริงไหม อ่านชื่อชิ้นงานถูกไหม — ต้องมีการวัดแบบ Phase 7 (นับ field ที่ตรงจริง ที่ละอัน) ถึงจะรู้คำตอบจริง

4. VRAM ขึ้นๆ ลงๆ ระหว่างเทรน — ทำไม

ระหว่างเทรน สังเกตว่า VRAM ไม่ได้นิ่งเท่ากันตลอด (บางช่วง 81GB บางช่วงพุ่งไป 94GB จากทั้งหมด 95GB — เกือบเต็มการ์ด) เหตุผล: ตัวอย่างเทรนแต่ละตัวไม่เท่ากัน — dataset นี้มีตัวอย่างพิเศษ 5 ตัวที่เรียกว่า "gridmaster" ใช้ภาพพร้อมกัน 2-4 ใบ (จำได้จากตอนที่ 2 ใหม่ เรื่อง MAX_LENGTH) พอเทรนไปเจอตัวอย่างพวกนี้ ก็ต้องใช้หน่วยความจำเยอะขึ้นชั่วคราว แล้วพอผ่านไปก็ลดกลับมาปกติ

เหมือนขับรถ — ทางราบใช้น้ำมันน้อย ทางขึ้นเขาใช้น้ำมันเยอะกว่าชั่วคราว ไม่ได้แปลว่ารถเสีย

5. ทำไม Qwen3-VL ช้ากว่า Qwen3.6 ถึง 2.9 เท่า ทั้งที่ตั้งค่าเหมือนกันหมด

t01 เทรนจบใน 29:23 นาที ส่วน t02 ใช้ไป 84:53 นาที — hyperparameter ทุกตัวตั้งเหมือนกันเป๊ะ (ตรวจสอบละเอียดแล้วในเอกสาร ไม่ใช่ความผิดพลาด) ความต่างนี้มาจากตัวสถาปัตยกรรมภายในของโมเดล เอง — Qwen3-VL กับ Qwen3.6 แม้จะเป็น MoE (Mixture of Experts) เหมือนกัน แต่การออกแบบภายใน (จำนวนชั้น, วิธีจัดการภาพ ฯลฯ) ต่างกัน ทำให้ความเร็วต่อ step ต่างกันได้มาก แม้ "ขนาดรวม" จะ ใกล้เคียงกัน

บทเรียน: ขนาดโมเดล (30B vs 35B) ไม่ได้ฟันธงเรื่องความเร็วเสมอไป การออกแบบภายในสำคัญกว่า เลขขนาดเปล่าๆ

6. ผลจริงของ Phase 7 — t02 แพ้ t01 ทุกตัวชี้วัด

	t01	t02	
JSON valid	90%	65.0%	
view ตรงเป๊ะ	70%	45.0%	
element recall	28.2%	20.6%	← ไม่ผ่านเกณฑ์ที่ตั้งไว้
element_type exact	87.5%	74.3%	
count exact	100%	66.7%	

นี่คือความล้มเหลวใหม่ — ไม่ใช่

คำถามที่ t02 ตั้งไว้แต่แรกคือ "Qwen3.6 หรือ Qwen3-VL อ่านแบบก่อสร้างไทยได้ดีกว่ากัน" ตอนนี้มีคำตอบแล้ว: Qwen3.6 ดีกว่า ทั้งแม่นยำและเร็วกว่า — นี่คือการที่ถูกต้องและมีค่า พอๆ กับถ้า t02 ชนะ เพียงแต่คำตอบคือ "อย่าเปลี่ยนไปใช้ Qwen3-VL" แทนที่จะเป็น "เปลี่ยนเถอะ"

เปรียบเทียบ: เหมือนทดลองยาสองตัว ถ้าตัว B ได้ผลแย่กว่าตัว A ก็ไม่ได้แปลว่าการทดลองล้มเหลว — มันแปลว่ารู้แล้วว่าไม่ควรใช้ยา B ซึ่งเป็นข้อมูลที่มีค่าจริง ดีกว่าไม่รู้อะไรเลย

สิ่งที่ยังเหลือทำ: Phase 7.5

แม้ผลจะค่อนข้างชัดแล้ว แต่ยังมีคำถามค้างๆ ตัวเลข 90%/28.2% ของ t01 (วัดเมื่อวันที่ 24 บน เครื่องเช่าคนละตัว) เชื่อได้เต็มที่ไหม เพราะโลบรารีที่เครื่องนั้นใช้ อาจเป็นคนละเวอร์ชันกับ เครื่องวันนี้ (ดูข้อ 2 ด้านบน — เพิ่งเจอมาสดๆ ว่าเวอร์ชันโลบรารีมีผลจริง) วิธีตัดข้อสงสัยนี้คือ วัด adapter ของ t01 ขึ้นบนเครื่องเดียวกับที่เพิ่งวัด t02 ถ้าได้ตัวเลขใกล้เคียงเดิม แปลว่า ผลเชื่อถือได้จริง ไม่ใช่ความบังเอิญของเครื่อง

7. ทำไม Phase 7.5 ต้องโหลดโมเดลใหม่ทั้งก้อน — LoRA adapter คืออะไรกันแน่

คำถามที่ระดมถามวันนี้: "ทำไมโหลดนาน" — คำตอบเชื่อมกับความเข้าใจเรื่อง LoRA ที่ควรรู้ให้ลึก ขึ้นอีกนิด

จำได้จากไฟล์ก่อนๆ ใหม่ว่า LoRA เหมือน "โพสต์อิทแปะทับ" — นี่คือเหตุผลที่แท้จริง: LoRA ไม่ได้เก็บโมเดลทั้งตัวใหม่ มันเก็บแค่ "ส่วนต่าง" เล็กๆ ที่บอกว่า "ถ้าเจอสถานการณ์แบบนี้ ให้ปรับคำตอบไปทางนี้นิดหน่อย" — ไฟล์ adapter ของ t01 มีขนาดแค่ ~7.5GB ในขณะที่โมเดลเต็มตัว (Qwen3.6-35B-A3B) หนักถึง ~70GB

ผลคือ: adapter อย่างเดียวใช้งานไม่ได้เลย ต้องมีโมเดลตั้งต้นตัวเต็มมาเป็นฐานเสมอ เหมือน โพสต์อิทแปะบนกระดาษเปล่าไม่มี ความหมาย ต้องมีสมุดเล่มเต็มให้แปะทับก่อน

วันนี้ตลอด Phase 4-7 เครื่องที่เช่าโฮลดแต่ **Qwen3-VL-30B-A3B** (โมเดลของ t02) ไม่เคยแตะ **Qwen3.6-35B-A3B** (โมเดลของ t01) เลย พอจะวัด t01 ซ้ำ (Phase 7.5) เลยต้องไปโฮลดโมเดลฐาน ตัวเต็มของ t01 มาใหม่ทั้งก้อนก่อน ถึงจะเอา adapter (ที่ดาวน์โหลดมาแล้ว) ไปแปะทับได้

8. คำศัพท์ใหม่วันนี้ (ต่อจากท้ายเล่มในตอนที่ 2)

คำศัพท์	ความหมายสั้นๆ
Controlled experiment (การทดลองแบบมีตัวควบคุม)	เปลี่ยนตัวแปรทีละตัวเท่านั้น ที่เหลือคุมให้เหมือนกันหมด เพื่อให้รู้แน่ชัดว่าผลต่างมาจากอะไร
Loss	ตัวเลขบอกว่าโมเดลทายผิดมากแค่ไหน ยิ่งต่ำยิ่งดี แต่บอกแค่ "ใกล้คำตอบทางคณิต" ไม่ได้บอกว่า "ถูกจริง"
A/B test	การเทียบสองตัวเลือกแบบคุมตัวแปร (ชื่อมาจากการเทียบแผน A กับแผน B)
Property (ของ class ในโปรแกรม)	ค่าที่อ่าน/เขียนผ่านชื่อ attribute ได้ แต่บางตัวถูกล็อกไว้ "อ่านได้อย่างเดียว" (readonly) เขียนทับไม่ได้
Library version drift	การที่ไลบรารีที่ใช้อยู่เปลี่ยนเวอร์ชันไปเรื่อยๆ ตามเวลา ทำให้โค้ดเก่าที่เคยใช้ได้ อาจใช้ไม่ได้ อีกต่อไป

9. เรื่องใหญ่ที่สุดของวันนี้ — ผลทั้งวันพลิกเพราะ "ค่าที่ไม่ได้ตั้ง" ไม่ใช่ "ค่าที่ตั้งผิด"

ค้นพบอะไร

หลัง Phase 7.5 เสร็จ ผมสั่งให้ตรวจแบบละเอียดมากอีกรอบ (agent 43 ตัวช่วยกันหาช่องโหว่) — เจอเรื่องใหญ่: **t01 (Qwen3.6)** ถูก เทรนด้วยภาพที่เบลอกว่า **t02 (Qwen3-VL)** ถึง ~14 เท่า โดยไม่มีใครตั้งใจ

ทำไมถึงเกิดขึ้นได้ทั้งที่เช็คละเอียดมากแล้ว

จำ "บั๊ก 2" ที่เจอตอน Phase 0.4 ได้ไหม (ตอนอ่านสคริปต์ก่อนเข้าเครื่อง) — ตอนนั้นเจอว่าถ้าไม่ตั้ง `resize="max"` ให้ชัดเจน โปรแกรมจะแอบย่อภาพเหลือ 512px แบบเนียนๆ แล้วแก้ปัญหานี้ไปแล้ว — แต่แก้แค่ฝั่งสคริปต์ของ t02 เท่านั้น ไม่มีใครนึกออกว่า ต้องย้อนกลับไปเช็คว่าสคริปต์ของ t01 (เขียนไว้ตั้งแต่เดือนก่อน) โดนปัญหาเดียวกันหรือเปล่า

ทำไมตารางเทียบ (parity table) ที่ทำไว้อย่างละเอียดถึงไม่จับได้: เพราะตารางนั้นเทียบแต่ "ค่าที่ตั้งใจตั้ง" (เช่น learning rate, batch size, จำนวน epoch) — ไม่ได้เทียบ "ค่าที่ปล่อยไว้ เป็นค่าเริ่มต้น (default) โดยไม่ได้พิมพ์อะไรเลย" และ `resize="max"` คือค่าแบบหลัง — ฝั่ง t02 พิมพ์ไว้ชัดเจน ฝั่ง t01 ไม่ได้พิมพ์อะไรเลย (เพราะเขียนไว้ก่อนจะรู้ว่ามีปัญหา) พอไม่มี บรรทัดให้ เทียบ ก็ไม่มีใครสังเกตว่ามันต่างกัน

เปรียบเทียบ: เหมือนเช็คสูตรอาหารสองสูตรว่าใส่เกลือ/น้ำตาล/พริกเท่ากันไหม (สิ่งที่จดไว้ในสูตร) แต่ลืมเช็คว่า "ไฟแรงแค่ไหน" ทั้งที่ ไม่มีใครเขียนระดับไฟลงในสูตรทั้งสองอัน (ใช้ไฟเริ่มต้นของเตา ที่ต่างกัน) — พอทำอาหารออกมารสไม่เหมือนกัน ก็เดาว่าเป็นเพราะ วัตถุดิบต่างกัน ทั้งที่จริงๆ คือไฟต่างกันตั้งแต่แรก

ยืนยันด้วยการวัดจริง ไม่ใช่แค่อ่านโค้ดแล้วเดา

โฮลดโมเดล Qwen3.6 ขึ้นมาจริง สร้าง "ตัวย่อภาพ" (collator) แบบเดียวกับที่ `train_qwen36.py` ใช้ เป๊ะ (ไม่ใช่ `resize=`) แล้วป้อน ภาพจริง 1 ใบเข้าไป — โปรแกรมพิมพ์ออกมาเองว่า:

Unsloth: Model does not have a default image size - using 512

แล้วนับได้ว่าเหลือแค่ ~266 visual token เทียบกับที่ t02 เทนด้วย ~3,796-5,100 token (ต่างกัน ~14 เท่า) — นี่คือการพิสูจน์ไม่ใช่การคาดเดา

ทำไมเรื่องนี้ถึงพลิกทุกอย่างที่สรุปไปก่อนหน้านี้

ทั้งวันสรุปไว้ว่า "Qwen3.6 อ่านแบบก่อสร้างไทยได้ดีกว่า Qwen3-VL" — ตอนนี้ต้องเปลี่ยนเป็น "Qwen3.6 ที่เห็นภาพเบลอกว่ามากยังทำได้ดีกว่า Qwen3-VL ที่เห็นภาพคมชัด" ซึ่งเป็นคนละ ข้อสรุปกัน — ข้อแรกตอบคำถามที่ตั้งไว้ตั้งแต่ต้น (ตระกูลไหนดีกว่า) ข้อหลังตอบไม่ได้ เพราะการ ทดลองไม่ยุติธรรมตั้งแต่ต้น (เหมือนให้นักวิ่งคนหนึ่งใส่ตุ้มถ่วงขาแล้วให้แข่งกัน ถ้าคนที่ใส่ตุ้ม ยังวิ่งชนะ ก็แปลว่าเก่งจริง แต่บอกไม่ได้ว่า "เร็วกว่ากันแค่ไหนถ้าไม่มีตุ้ม")

ทางเลือกที่เสนอ + สิ่งที่เหมาะสมเลือก

เสนอ 2 ทาง: (A) หยุดตรงนี้ ยอมรับผลพร้อมค่าเตือนกำกับชัดเจน หรือ (B) เทน t01 ใหม่ด้วย `resize="max"` ให้เห็นภาพคมชัดเท่า t02 แล้ววัดใหม่ให้ยุติธรรมจริง (เสียเวลา+เงินเพิ่ม) **เหมาะสมเลือก (A)** — รับผลตามที่มีพร้อมค่าเตือนกำกับไว้ในเอกสารทุกจุด ไม่เทรนซ้ำ

บทเรียนที่บันทึกไว้ถาวร

เพิ่มเป็น Lesson 12 ใน `rule_of_tune.md`: ตารางเทียบตัวแปรสำหรับการทดลองแบบ A/B ต้องเขียน ทุก argument ที่ส่งเข้าฟังก์ชัน/class ที่ใช้ร่วมกัน ไม่ใช่แค่ค่าที่ตั้งใจเลือก — ค่าที่ปล่อยไว้ เป็นค่าเริ่มต้นก็เป็นตัวแปรเหมือนกัน และเพราะไม่มีบรรทัดให้เทียบ จึงหลุดสายตาได้ง่ายที่สุด รอบต่อไปต้องไล่เทียบทุก argument จริงจากซอร์สโค้ด ไม่ใช่แค่รายการที่จำได้ว่าเปลี่ยนอะไรไปบ้าง

ตอนที่ 4 (2026-07-29) — คินนี่เจอว่า "ของที่เราภูมิใจที่สุด" อาจไม่เคยถูกทุนเลย

ไฟล์นี้ต่อจากตอนที่ 3 ด้านบน (ศัพท์พื้นฐาน — GGUF, mmpoj, LoRA, VRAM, MAX_LENGTH, loss, visual token, collator, merge — อธิบายไว้ในตอนที่ 2 กับ 3 แล้ว ที่นี้อ้างถึงเฉย ๆ ไม่อธิบายซ้ำ)

คินนี่ต่างจากทุกคิน — สองคินก่อนคุยกันเรื่อง "ทุนโมเดลยังงัยให้สำเร็จ" คินนี่เอาโมเดลที่ทุนสำเร็จแล้วมาใช้งานจริงเต็มรูปแบบครั้งแรก แล้วพบว่ามันใช้ไม่ได้ พอไล่หาสาเหตุ กลับเจอเรื่องที่ใหญ่กว่าที่คิดไว้มาก

สรุปสั้นสุดก่อนอ่านยาว: เรามีเหตุผลหนักแน่นที่จะสงสัยว่า **ไฟล์ GGUF ของ t01** ที่เราใช้มาตลอด อาจเป็นโมเดลเปล่าที่ไม่มีการทุนอยู่ในนั้นเลย — ด้วยบั๊กตัวเดียวกัน กับที่ทำให้ t02 แปลง GGUF ไม่สำเร็จเมื่อวันที่ 28 ต่างกันแค่ **t02** ถูกจับได้ ส่วน **t01** หลุดรอด เพราะด้านตรวจของ t01 อ่อนเกินไป · ยังไม่ยืนยัน 100% แต่ทดสอบได้ฟรีใน 20 นาที

1. เกิดอะไรขึ้นคินนี่ — เรียงตามเวลา

- เอา t01 (ทุนไว้วันที่ 21-24 แปลงเป็น GGUF เก็บในเครื่อง) มารันถอดแบบบ้าน 01 ต่อ
- ได้ผลมา 4 หน้าใหม่ (หน้า 03-06)
- มะขามเปิดดูแล้วบอก "ผลโคตรห่วย"
- ผมเทียบกับเฉลยจริงทั้ง 6 หน้า → มะขามพูดถูก
- ไล่หาสาเหตุ → เจอข้อสงสัยที่ใหญ่กว่าที่คาดไว้มาก

2. "ห่วย" ห่วยยังงัย — แต่ต้องพูดให้แฟร์ก่อน

สิ่งที่โมเดลทำได้ดีจริง (ต้องพูดถึงด้วย ไม่งั้นวินิจฉัยผิด)

หน้า	ทำได้ดี
02	sheet range ถูก 5/5 · เลขหน้าเริ่มต้นถูก 8/8 (1/16/24/31/35/36/46/56)
04	เลข มอก. ครบ 8/8 · สเปกก่ออิฐ RB6 ตรงค่าต่อคำ
05	หมายเหตุในชั้ดถูก 6/6 เกือบค่าต่อคำ · grid แนวตั้งถูกเป๊ะ
06	ระดับพื้นถูก 7/8 · เจอห้องครบ 8/8 · SLOPE 6° และกันสาด +3.75 ถูก

ในฐานะ "เครื่องอ่านตัวหนังสือไทยบนแบบก่อสร้าง" มันใช้ได้จริง — อ่านออก อ่านแม่นยำ ที่สำคัญมาก เพราะมันบอกว่าปัญหาไม่ได้อยู่ที่ "ตาไม่ดี"

สิ่งที่ฟัง — และฟังในแบบที่แปลกมาก

ก. ไม่ตอบตาม schema ที่เทรนมาเลยสักหน้า (0/6)

```
เฉลยใช้ key:  "png" / "doc_page" / "discipline" / "sheet_code"
AI ตอบด้วย:  "page" (หน้า 01)
              "page_number" (หน้า 02, 04)
              "page_number" + "sheet_id" (หน้า 03, 05, 06)
              † สามแบบสลับไปมา ไม่มีแบบไหนตรงเลย
```

ข. หลอนของที่ไม่มีอยู่ (หน้า 06 นรกสุด)

เจเลยมี 17 ชั้น AI ตอบมา 59 ชั้น — ส่วนเกินคือเสา 12 ต้น + ประตู 8 + หน้าต่าง 4 ที่ไม่มีอยู่บนแบบแปลนนั้นเลย และ mark ที่มันแปะให้คือ A-05 A-07 A-09

A-05 ไม่ใช่ชื่อเสา มันคือเลขชี้ด (A-05 = แผ่นรูปด้าน, A-07 = แผ่นรูปตัด) บนแบบมีข้อความประมาณ "ดูรายละเอียดที่แผ่น A-05" AI เห็นแล้วนึกว่าเป็นชื่อของเสา

เหมือนอ่านหนังสือเจอ "ดูหน้า 57" แล้วสรุปว่า "ตัวละครชื่อคุณหน้า 57" — อ่านตัวหนังสือออก แต่ไม่เข้าใจว่าตัวหนังสือนั้นทำหน้าที่อะไร

ค. แต่งตัวเลข geometry ขึ้นมาเอง

เจเลย grid แนวนอน: D=0, C=4.0, B=6.0, A=9.5
AI ตอบ: A=0, B=1.5, C=5.0, D=9.5
↑ ตัวอักษรกลับด้าน ระยะในผัดหมด ถูกแค่ผลรวม 9.5

และหน้า 06 มันสร้าง grid_table ที่ให้ตัวเลข 1 ค่าต่อจุดตัด grid ซึ่งไม่ใช่ปริมาณที่มีอยู่จริงในโลก (จุดตัดไม่มี "ความยาว") — เอาเลขจริงบนแบบ (7.0, 9.5, 4.0, 3.5) มาสับใส่ช่องมั่ว ๆ prompt เขียนว่า "span ต้องมาจาก grid table" → ทุก span ที่คำนวณต่อจากนี้ ผิดหมด

ง. ชัดแย้งกันเอง

F6 = ["C-2", "C-3", "D-3", "D-2"] → "เตรียมอาหาร"
F7 = ["C-3", "C-2", "D-2", "D-3"] → "นอน 2"
↑ สักแต่แย้งกันเป๊ะ แต่บอกว่าเป็นคนละห้อง

จ. ชั้นความมั่นใจหายทั้งหมด — เจเลยมี confidence_score + confidence_flags บอกว่าจุดไหนอ่านไม่ชัด (เช่น "ตัวหนังสือเล็กเกิน ต้อง zoom ตรวจ") AI ตอบมา 0 ครั้งใน 6 ไฟล์ → เราไม่รู้เลยว่าตรงไหนเชื่อได้ตรงไหนเชื่อไม่ได้

ฉ. สลับวัสดุจนกระเทย BOQ (หน้า 04)

เจเลย: "ผนัง" ก่ออิฐ "มอญ"
AI: "ฝ้าเพดาน" ก่ออิฐ "มวลเบา"
↑ ผิดทั้งหมดงานและชนิดวัสดุ → ถ้าเอาไปคิดราคาคือผิดสองชั้น

3. 🔑 เบาะแสสำคัญ: "ฟังในแบบที่ผิดธรรมชาติของโมเดลที่ทุนมาแล้ว"

ตรงนี้คือหัวใจ — อ่านซ้ำ ๆ

ข้อเท็จจริงที่ทำให้ทุกอย่างเปลี่ยน

หน้า 03-06 คือ training data ของ t01 เอง — โมเดลเคยเห็นหน้าพวกนี้พร้อมเจเลย ตอนเทรน 3 รอบ (epoch)

แล้วทำไมถึงแปลก

ในการ fine-tune มีสิ่งหนึ่งที่โมเดลเรียนได้เร็วที่สุดเสมอ คือ "รูปแบบของคำตอบ" (format) — ชื่อ key ต้องใช้อะไร โครงสร้าง JSON หน้าตาอย่างไร โมเดลจับได้ภายในไม่กี่ร้อยก้าวแรกของการเทรน

เนื้อหายากกว่ามาก (ต้องอ่านเลขบนแบบให้ถูก) — อันนั้นพลาดได้ ไม่แปลก

แต่คืนนี้เราเจอ ตรงกันข้ามเป๊ะ:

เนื้อหา (ของยาก): ทำได้ดี — เลข มอก. 8/8, ระดับพื้น 7/8, ห้อง 8/8
format (ของง่าย): ฟังหมด — 0/6 หน้า และใช้ 3 แบบสลับกันเอง

นี่ผิดธรรมชาติ ถ้าโมเดลถูกทุนมาจริง format ต้องเป๊ะก่อนเนื้อหาเสมอ

สิ่งที่ผลลัพธ์แบบนี้เหมือนที่สุดคือ...

โมเดลเปล่า ๆ ที่ไม่เคยถูกทูน แล้วมีคนยื่น prompt อธิบาย schema ให้อ่าน — มันจะพยายามทำตาม prompt (จึงมี `views / pattern` ตามที่ prompt สั่ง) แต่จะไม่รู้จัก key อย่าง `png / doc_page` ที่มีอยู่แค่ในเจลอ ไม่ได้เขียนใน prompt และนั่นคือสิ่งที่เราเห็นปะ ๆ

4. 🔑 หลักฐานที่ทำให้สงสัยว่า LoRA ไม่ได้อยู่ใน GGUF

ผมไปเปิดโค้ดจริงดู เจอ 2 อย่างที่ต่อกันเป็นเรื่องเดียว

หลักฐาน A — t01 ใช้กลไก LoRA แบบเดียวกับที่ทำ t02 พัง

เปิด `train_qwen36.py` บรรทัด 152 มีคอมเมนต์ที่เราเขียนไว้เองตั้งแต่วันที่ 21:

```
lora_dropout=0, # ต้องเป็น 0 - MoE ParamWrapper ของโมเดลนี้ไม่รองรับ dropout!=0
```

คำว่า `ParamWrapper` สำคัญมาก — มันคือกลไกพิเศษของ peft ที่ใช้ปะ LoRA ลงบน "ผู้เชี่ยวชาญ" (experts) ของโมเดลแบบ MoE

และเมื่อคืนวันที่ 28 เราพิสูจน์ไปแล้วด้วยมือตัวเองว่า:

```
merge_and_unload() (ฟังก์ชันที่ใช้รวม LoRA เข้ากับโมเดลก่อนแปลงเป็น GGUF)
```

รวม LoRA แบบ `ParamWrapper` บน MoE ไม่ถูกต้อง — เราวัดค่าจริงทั้ง tensor 201 ล้านค่า พบว่าเปลี่ยนไปแค่ 0.67% ทั้งที่ควรเปลี่ยนมากกว่านั้นเยอะ
= โครงสร้างผ่าน ไม่ error แต่พฤติกรรมเกือบกลับไปเป็นโมเดลเปล่า

t01 ใช้ฟังก์ชันเดียวกัน กับโมเดล MoE เหมือนกัน กับ `ParamWrapper` เหมือนกัน

หลักฐาน B — ด้านตรวจของ t01 อ่อนจนโมเดลเปล่าก็ผ่าน

ตอนแปลง GGUF เรามีด้านตรวจกัน "merge พังเจียน" อยู่แล้ว เปิดดู `export_gguf.py` บรรทัด 107:

```
_looks_tuned = _pred.lstrip().startswith("{") and any(k in _pred[:400]
for k in ('png', 'views', 'pattern', 'doc_page'))
```

แปลว่า: "ถ้าคำตอบขึ้นต้นด้วย { และ มีค่าใดค่าหนึ่งใน 4 ค่านี้ → ถือว่าทูนแล้ว"

ปัญหาคือ prompt ของเราเองสั่งไว้ตรง ๆ ว่า:

```
...emit one entry per view in "views"
Each view carries its own "pattern": plan, section, schedule, ...
```

→ prompt บอกคำว่า `views` และ `pattern` ให้โมเดลอยู่แล้ว

→ โมเดลอะไรก็ตามที่ทำตาม prompt ได้ ก็จะมีค่าสองค่านี้ออกมา

→ โมเดลที่ไม่เคยทูนเลยก็ผ่านด้านนี้สบาย ๆ

ตอนแปลง GGUF วันที่ 24 ด้านนี้ขึ้น ✓ สีเขียว แล้วเราก็เชื่อ — แต่จริง ๆ มันไม่ได้พิสูจน์อะไรเลยแม้แต่นิดเดียว

ทำไม t02 ถึงถูกจับได้ แต่ t01 หลุด

t02 (วันที่ 28): ด้านเทียบ "ชื่อ key จริง" → เจอ `page_number/title/project`
แทนที่จะเป็น `png/doc_page/...` → ❌ หยุดทันที ✅ ทำงานถูกต้อง

t01 (วันที่ 24): ด้านเทียบแค่ "มีคำว่า `views` หรือ `pattern` ไหม"

→ ✓ ผ่าน (แต่ผ่านเพราะ prompt สั่งไว้ ไม่ใช่เพราะทูนสำเร็จ)

บักเดียวกัน ต่างกันแค่คุณภาพของด้านตรวจ

 วิธีพิสูจน์ — ถูกและเร็ว ทำพุงนี้ได้เลย

เอา โมเดล Qwen3.6 ตัวเปล่า (base, ไม่ทูน) มารันหน้า O3 ด้วย prompt เดียวกัน

- ถ้าผลออกมาหน้าตาเหมือนของ t01 → ยืนยัน: LoRA ไม่ได้อยู่ใน GGUF

- ถ้าผลออกมาแยกว่าชัดเจน → LoRA อยู่จริง ต้องไปดูสาเหตุอื่น

ใช้เวลา ~20 นาที ฟรี ไม่ต้องเช่าอะไร และต้องทำก่อนใช้เงินกับอะไรทั้งสิ้น

5. สาเหตุรองอีก 2 ข้อ (จริงทั้งคู่ แต่ไม่ใช่ตัวหลัก)

5.1 เทรนกับตอนใช้ เห็นภาพคนละความละเอียด 14 เท่า

ล็อกของ llama-server บอกจำนวน visual token ที่ป้อนจริง:

```
2048 + 1748 = 3,796 token/ภาพ ← ตอนใช้งานคั่นนี้
~266 token/ภาพ ← ตอนเทรน (บัก collator ย่อเหลือ 512px)
ต่างกัน 14.3 เท่า
```

เปรียบเทียบ: สอนเด็กอ่านแผนที่ด้วยแผนที่ขาวดำเบลอทั้งหมด เด็กทำข้อสอบแผนที่เบลอได้ 90 คะแนน

วันสอบจริงเอาแผนที่สีความละเอียดสูงกว่า 14 เท่าให้ดู — เด็กไม่ได้โง่งง

แต่ "ไม่เหมือนที่เคยฝึกมาเลย"

llama.cpp เดือนเราเองในล็อกด้วยซ้ำ (แต่ไม่มีใครอ่าน):

```
W load_hparams: Qwen-VL models require at minimum 1024 image tokens to function correctly
W load_hparams: if you encounter problems with accuracy, try adding --image-min-tokens 1024
```

บทเรียนย่อย: ค่าเดือนตอนโหลดโมเดลมักถูกมองข้ามเพราะขึ้นปนกับข้อความอีกเป็นร้อยบรรทัด — นี่คือ pattern เดิมซ้ำรอบที่ 2 แล้ว (รอบแรกคือ collator พิมพ์ `Model does not have a default image size - using 512` แล้วไม่มีใครเห็น เหมือนกันเป๊ะ)

5.2 การบีบอัด (quantization) ที่เจ้าของโมเดลเดือนเองว่าอย่าทำ

quantization คืออะไร: โมเดลคือตัวเลขหลายพันล้านตัว ปกติเก็บด้วยความละเอียดสูง (bf16) Quantization = ลดความละเอียดลง ให้ไฟล์เล็กพอรันบนเครื่องเล็ก Q4_K_M → จาก 70GB เหลือ 21GB

เหมือนบีบอัดรูป JPEG — บีบนิดหน่อยแทบไม่เห็นต่าง บีบเยอะ ๆ เริ่มเห็นรอยเบลอเป็นตาราง

เอกสาร t01_workflow.md เขียนไว้เองว่า Unsloth (คนทำโมเดล) เดือนว่าตระกูลนี้ "quantize แล้วคุณภาพตกผิดปกติ" — ซึ่งเป็นเหตุผลที่เราบังคับเทรนด้วย bf16 ห้ามใช้ 4-bit

แต่พอถึงตอนเอามาใช้ เรากลับบีบเป็น Q4_K_M (เพราะจำเป็น — โนตบุ๊ก VRAM 8GB รับ 70GB ไม่ไหว) แล้วไม่ได้กลับไปวัดซ้ำว่าบีบแล้วยังแม่นอยู่ไหม

→ เราละวังเรื่องนี้ตอนเทรน แต่ลืมระวังตอนใช้

6. บทเรียนเรื่อง "ตัวเลข 90% ที่เราเชื่อมาตลอด"

เส้นทางรันโมเดลมี 2 เส้น ไม่ใช่เส้นเดียว

```
เส้นทาง A — ตอนวัดผล (eval_fields.py)
โมเดล bf16 เต็ม 70GB + LoRA adapter แบบ runtime (ไม่ merge)
→ ได้ 90% JSON valid, 28.2% element recall

เส้นทาง B — ตอนใช้จริงในเครื่องมะขาม
merge LoRA เข้าไป (← ขั้นที่สงสัยว่าพัง) แล้วบีบเป็น Q4_K_M 21GB
→ ไม่เคยวัดความแม่นยำ
```

ผมเปิด `eval_fields.py` บรรทัด 29 ดู เขียนไว้ว่า `load_in_4bit=False` = วัดด้วยโมเดลเต็ม ไม่บีบอัด และไม่ผ่านขั้น merge

แล้ว GGUF เคยถูกทดสอบไหม — เคย แต่ทดสอบผิดเรื่อง

Phase 10 ของ t01 (วันที่ 25) ทดสอบ GGUF จริง เกณฑ์ผ่านคือ:

- ☒ ตอบเป็น JSON ใหม่ → ผ่าน
- ☒ ไม่ค้าง ไม่ crash ใหม่ → ผ่าน
- ☒ เนื้อหาถูกไหม → ไม่เคยตรวจ

เอกสารเขียนเตือนไว้เองด้วยซ้ำว่า "don't read this as proven complete" แต่พอเวลาผ่านไป ทุกคน (รวมผม) จำได้ว่า "Phase 10 ผ่านแล้ว"

เปรียบเทียบ: ทดสอบรใหม่แค่ว่า "สตาร์ทติดไหม เข้าเกียร์ได้ไหม" → ติด → ประกาศว่า "รถพร้อมใช้" โดยไม่เคยขับดูว่าเลี้ยวถูกทางไหม เบรกอยู่ไหม

⚠ ข้อสำคัญที่ต้องไม่เข้าใจผิด

ตัวเลข 90% / 28.2% ยังใช้ได้อยู่ — สำหรับ LoRA adapter สิ่งที่ยังไม่ยืนยันคือ ไฟล์ GGUF ต่างหาก

งานทุนของมะขามไม่ได้สูญเปล่า — adapter ยังอยู่ครบนบน HuggingFace ที่ฟังคือขั้นตอนแปลงไฟล์ไปใช้งาน ไม่ใช่ตัวการทุน

7. ทำไมมันช้า — เลขจริงและเหตุผล

อย่าง	ความเร็ว
อ่านภาพ + prompt	3,924 token @ ~28 t/s = ~140 วินาที
เขียนคำตอบ	3.46 token/วินาที
ถอดเต็ม 1 หน้า	~13-21 นาที
classify 1 หน้า	~2.4 นาที

เหตุผล — เลขคณิตง่าย ๆ

```
ไฟล์โมเดล    21 GB
VRAM          8 GB    → ยัดลงการจ่อได้แค่ ~1 ใน 3 ที่เหลือรันบน CPU
```

ล็อกยืนยันตรง ๆ: `failed to fit params to free device memory ... abort`

เกร็ดที่น่าสนใจ: GPU แสดง 100% แต่กินไฟแค่ 33.7W (การ์ดตัวนี้กินได้มากกว่านี้เยอะ) — แปลว่าการ์ดไม่ได้ทำงานหนัก มันแค่รอ CPU อยู่ตลอด CPU คือคอขวดตัวจริง

ทำไม classify เร็วกว่าถอดเต็ม 7 เท่า ทั้งที่ดูภาพเดียวกัน

เพราะเวลาส่วนใหญ่หมดไปกับ "การเขียนคำตอบ" ไม่ใช่ "การดูภาพ"

```
classify:  ดูภาพ 140 วิ + เขียน ["plan","schedule"] ~10 token = 3 วิ    → ~144 วิ
ถอดเต็ม:  ดูภาพ 140 วิ + เขียน JSON ยาว ~3,800 token = 1,100 วิ    → ~1,250 วิ
```

ที่ 3.46 token/วินาที ทุก token ที่ต้องเขียนคือเวลาจริงที่เสียไป → นี่คือเหตุผลที่แผน 2 phase (สแกนไวก่อน แล้วถอดเฉพาะหน้าที่ต้องการ) ช่วยได้จริง — ดูภาคผนวก A ว่า `extract_house01_local.py` ทำ 2-phase นี้จริงยัง

8. บั๊กที่ผมทำเองคืนนี้ — บทเรียน "เจียบ ≠ ค้าง"

ผมตั้ง timeout ไว้ 1,200 วินาที (20 นาที) โดยคำนวณจากหน้า 01-02 พอเจอหน้า 03 ที่ใช้ 1,256 วินาที → ผมตั้งงานที่กำลังทำอยู่จริงทั้ง

วิธีที่รู้ว่าไม่ได้ค้าง: ดู `nvidia-smi` → GPU ยังทำงาน 100% (ถ้าค้างจริงจะเป็น 0%)

บทเรียน 1: "ไม่มี output" ≠ "ค้าง" — โมเดลที่กำลังคิดอยู่ก็เจียบเหมือนกัน ต้องดูตัวชี้วัดอื่น (GPU util, ไฟล์ที่โตขึ้น) ก่อนสรุปว่าตาย ถ้าผมเชื่อความเจียบ ผมจะไปแก้ผิดจุดทั้งคืน

บทเรียน 2: หน้า 01-02 คือหน้าปกกับสารบัญ = เบาที่สุดในเล่ม เอาเวลาจากสองหน้านั้นมาตั้งเกณฑ์ให้ทั้งเล่ม → ผิดตั้งแต่ต้น ตัวอย่างที่ใช้ตั้งเกณฑ์ต้องครอบคลุมกรณีหนักสุด ไม่ใช่กรณีที่บังเอิญเจอก่อน

9. ⚠ ข้อควรระวังในการตีความผลคืนนี้ (สำคัญ อย่าข้าม)

อย่าเพิ่งด่วนสรุปว่า "โมเดลนี้ใช้ไม่ได้" เพราะการทดสอบคืนนี้เองก็ไม่แฟร์กับโมเดล:

- หน้า 03/04/05 = index / notes / material_list / site_plan ซึ่งเป็น pattern ที่แผนใหม่ (KEEP_PATTERNS) คัดออกอยู่แล้ว → เามาตัดสินโมเดล = ตัดสินจากงานที่มันจะไม่ถูกใช้ทำ
- ยังไม่มีหน้าโครงสร้าง (S-*) ถูกถอดเลยสักหน้า — หน้า 07-11 แค classify → กฎใหญ่สุดของ prompt ("แยกหลักบน/ล่างเสมอ") ยังไม่มีหลักฐานทั้งฝั่งดีและแย → เรายังไม่รู้เลยว่ามันถอดหลักได้แค่ไหน ซึ่งเป็นงานจริงที่เราต้องการ
- การ classify เองก็ยังไม่แม่น — หน้า 07/08/09 ถูกจัดเป็น section แต่เจียคือ side_profile (อ่านรูปด้านเป็นรูปตัด) และบอกว่ามี 5 view ทั้งที่มีจริง 2 → ถ้า classify ผิด หน้าที่ควรถอดอาจถูกข้าม หรือหน้าที่ควรข้ามถูกถอด

10. สรุปบทเรียนคืนนี้ — 6 ข้อที่ควรจำ

- "ทุนสำเร็จ" ≠ "ใช้งานได้" ระหว่างสองอย่างนี้มีขั้น merge → quantize → เปลี่ยนโปรแกรมรัน อีกหลายขั้น ทุกขั้นทำให้ผลเปลี่ยนได้ และต้องวัดซ้ำทุกขั้น
- ด้านตรวจที่ผ่านง่ายเกินไป แยกว่าไม่มีด้านเลย เพราะมันให้ความมั่นใจปลอม `export_gguf.py:107` ขึ้น ✓ เขียนตั้งแต่วันที่ 24 แล้วเราก้เชื่อมาตลอด ด้านตรวจต้องถามว่า "ถ้าของพังจริง ด้านนี้จะจับได้ไหม" — ถ้าคำตอบคือ "ไม่" ด้านนั้นไร้ค่า
- สังเกต "รูปแบบของความผิดพลาด" ไม่ใช่แค่ "ผิดมากผิดน้อย" คืนนี้เบาะแสสำคัญที่สุดไม่ใช่ "ผิดเยอะ" แต่คือ "ผิดในของง่าย ถูกในของยาก" ซึ่งผิดธรรมชาติ → นำไปสู่การเจอสาเหตุจริง ถ้าดูแค่คะแนนรวม จะไม่มีวันเจอ
- ตัวเลขความแม่นยำต้องมาพร้อมคำถามว่า "วัดจากอะไร" 90% ของเรารวัดจาก bf16 + adapter แบบไม่ merge — ไม่ใช่ของที่รันจริงในเครื่อง
- input ตอนใช้ต้องเหมือน input ตอนเทรน (ศัพท์ทางการ: train/inference mismatch) ต่างกัน 14 เท่าเมื่อไหร่ ความรู้ที่เทรนมาใช้ไม่ได้ทันที
- คำเตือนที่โปรแกรมพิมพ์ออกมา ต้องอ่าน — เจอ pattern นี้ครบ 2 รอบแล้ว ครั้งหน้าให้ไล่อ่านคำเตือนตอนโหลดโมเดลทุกครั้ง ก่อนรันงานยาว

11. พรุ่งนี้ทำอะไรต่อ — เรียงตามลำดับ

● อันดับ 1 (ฟรี, ~20 นาที) — พิสูจน์ว่า LoRA อยู่ใน GGUF ใหม่

รันหน้า 03 ด้วย base Qwen3.6 GGUF ตัวเปล่า เทียบกับผลของ t01 **ต้องทำก่อนตัดสินใจอะไรทั้งหมด** เพราะถ้าคำตอบคือ "ไม่อยู่" ทางเลือกอื่นเปลี่ยนหมด

● อันดับ 2 (ฟรี, ไม่กี่นาที) — แก่ด้านตรวจใน `export_gguf.py`

ให้เทียบชื่อ key จริงแบบที่ t02 ใช้ ไม่ใช่แค่หาคำว่า `views / pattern` (ใช้ได้ทั้งกับ t01 และทุกรอบต่อไป)

● อันดับ 3 — ค่อยเลือกทางแก้ ตามผลของอันดับ 1

ถ้าผลออกมาว่า...	ทางที่ควรทำ
LoRA ไม่อยู่ใน GGUF	ต้องหารีธี merge ใหม่ (มักเดียวกับ t02 ที่ยังแก้ไม่ได้) หรือ เลิกใช้ GGUF ไปรัน bf16+adapter บนเครื่องเช่าแทน — เส้นทางหลังนี้รู้แน่ว่าได้ 90%
LoRA อยู่จริง	ปัญหาอยู่ที่ resolution/quantization → ลอง <code>--image-min-tokens</code> ก่อน (ฟรี) ถ้าไม่ดีขึ้นค่อยพิจารณาเทรนใหม่ด้วย <code>resize="max"</code>

— สิ่งที่ยังไม่ควรทำ

- อย่าเพิ่งเช่า GPU — การเช่าแก้ได้แค่ "ช้า" แก่ "ห่วย" ไม่ได้ ถ้า LoRA ไม่อยู่ใน GGUF จริง เข้าไปก็จะได้ผลเหมือนเดิม แค้นห่วยเร็วขึ้น 10 เท่าและเสียเงินฟรี
- อย่ารันทัน 01 ต่อ จนกว่าจะรู้ผลอันดับ 1 ไม่จันได้ข้อมูลผิดเต็มโพลเดอร์

12. คำถามที่ยังไม่มีคำตอบ

- LoRA อยู่ใน GGUF ของ t01 หรือไม่ ← **คำถามที่สำคัญที่สุด**
- ถ้าไม่อยู่ — แล้วผลลัพธ์ทั้งหมดที่เคยตัดสินใจจาก GGUF ตัวนี้ ต้องรื้อก็อย่าง
- โมเดลถอดเหล็กเสริมได้ดีแค่ไหน ← ยังไม่เคยทดสอบเลย (ไม่มีหน้า S-* ถูถอด) ทั้งที่นี่คืองานจริงที่เราต้องการที่สุด
- ข้อสังเกตที่น่าสนใจมาก: t02 (Qwen3-VL) เทรนด้วยภาพ **3,796 token** ซึ่งตรงกับที่ `llama.cpp` ป้อนให้พอดี — ถ้าแปลง GGUF ได้ มันอาจทำงานในเครื่องได้ดีกว่า t01 ด้วยซ้ำ (แต่ตอนนี้แปลงไม่ได้ด้วยบัก merge ตัวเดียวกัน — ซึ่งตอนนี้กลายเป็นบักที่บล็อกทั้งสองรอบ ไม่ใช่แค่ t02) ยังไม่ได้พิสูจน์

ภาคผนวก A — ไฟล์ทุกไฟล์ใน tune_ai/t01/ ทำหน้าที่อะไรบ้าง

รวบรวมจากการเปิดอ่านไฟล์จริงในโฟลเดอร์ `tune_ai/t01/` ของ repo `Training` (ไม่ใช่การเดา) ตอนนี้นำแบ่งเป็น 4 ส่วน: เอกสารประจำรอบ, ชุดข้อมูล+สคริปต์เทรน (`data_before_tune/`), สคริปต์ถอดแบบจริงบนคอมมะขาม, และผลลัพธ์การถอดแบบ (`ผล/`)

A.1 เอกสารประจำรอบ (ระดับโฟลเดอร์ t01/)

ไฟล์	หน้าที่
<code>t01_workflow.md</code>	คู่มือหลักของรอบ t01 ทั้งหมด — Phase 0 ถึง Phase 11 (เช็คก่อนเข้า → เข้า → อัปโหลด → เทรน → วัดผล → export GGUF → verify → download → รันโลคอล → ทำลาย instance) แต่ละ Phase มีบันทึกจริงว่าทำอะไร เจอบั๊กอะไร แก้ยังไง (ไม่ใช่แค่แผนลอยๆ) — สถานะปัจจุบัน: Phase 0-10 เสร็จแล้ว, Phase 11 (ทำลาย instance) ยังไม่ทำเพราะต้องรอคำสั่งชัดเจนจากมะขามเท่านั้น ชื่อเดิมคือ <code>RETUNE_WORKFLOW.md</code> (เปลี่ยนชื่อ 2026-07-28 ให้บอกรอบชัดเจน กัน t01/t02 ปนกัน)
<code>extract_house01_local.py</code>	สคริปต์ Python ที่ใช้โมเดล t01 ที่ทุนแล้วจริง (GGUF ในเครื่องมะขาม) ถอดแบบบ้านเล็ก_1ชั้น_01 ทั้ง 61 หน้า ผ่าน <code>llama-server</code> — รายละเอียดเต็มอยู่ในภาคผนวก B ด้านล่าง (มีทั้ง prompt classify และ prompt extract อยู่ในไฟล์นี้)

A.2 data_before_tune/ — ชุดข้อมูลเทรน + สคริปต์เทรนบนเครื่องเช่า

โฟลเดอร์นี้คือสิ่งที่อัปโหลดขึ้นเครื่องเช่า Vast.ai ทั้งก่อน (~240MB ตอนแรก) ตามที่อธิบายไว้ในตอนที่ 1-2 ด้านบน

ไฟล์/โฟลเดอร์	หน้าที่
<code>README.md</code>	สเปกของชุดข้อมูล — เลือกใช้โมเดลไหน (Qwen3.6-35B-A3B) ทำไม, เข้าเครื่องแบบไหน, ทำตามขั้นตอนไหน, มีอะไรในโฟลเดอร์, และ "การตัดสินใจที่ฝังอยู่ในชุดนี้" (ทำไมตัด <code>specs{}</code> ทั้ง, ทำไมตัดบันทึกการทำงานออก, ทำไม instruction ยาว ~1,020 token, ทำไมแบ่ง val ตามหลังไม่ใช่ตามหน้า, ทำไม 5,120 visual token) — ก็คือ "ทำไมชุดข้อมูลถึงหน้าตาแบบนี้" ทั้งหมด
<code>build_dataset.js</code>	สคริปต์ Node.js ที่ประกอบ <code>train.jsonl / val.jsonl</code> จริงจากต้นทาง <code>json_แก้ไขแล้ว/</code> — รับเข้าได้ผลเหมือนเดิมทุกครั้ง (deterministic) ควบคุมด้วย <code>CFG</code> (เช่น <code>DROP_CROSS_PAGE_SPECS</code> , <code>STRIP_WORKLOG</code> , <code>PROMPT_MODE</code> , <code>VAL_HOUSES</code>) เป็นที่มาจริงของ <code>PROMPT_SHORT</code> (prompt ที่ใช้สอนโมเดล — ดูภาคผนวก B)
<code>train.jsonl</code>	ชุดตัวอย่างสอนโมเดลจริง (บ้าน 01/02/04/05 รวม 315 บรรทัด ล่าสุด — เริ่มต้น 226 ก่อนเพิ่มบ้าน 05) แต่ละบรรทัดคือ 1 ภาพ + คำตอบ JSON ที่ถูกต้อง
<code>val.jsonl</code>	ชุดตัวอย่างสอบ (บ้าน 03, 88 บรรทัด) — โมเดลไม่เคยเห็นตอนเทรน ใช้วัดผลจริง
<code>images/</code>	ไฟล์ภาพ PNG ต้นทาง (310+ ไฟล์ ขนาด 3309×2339) ที่ <code>train.jsonl / val.jsonl</code> อ้างอิงถึง
<code>onstart.sh</code>	สคริปต์ที่วางในช่อง "On-start script" ตอนสร้าง instance บน Vast.ai — รันอัตโนมัติตอนเปิดเครื่อง: ตั้งค่า <code>TORCH_CUDA_ARCH_LIST</code> สำหรับการ์ด Blackwell, ติดตั้ง <code>triton / unsloth / trl / peft / accelerate / bitsandbytes / pillow</code> ผ่าน pip
<code>verify_env.py</code>	เช็คสภาพแวดล้อมเครื่องเข้าก่อนอัปโหลด dataset จริง — GPU/CUDA compute capability, VRAM พอไหม (≥74GB), Unsloth/Triton โหลดได้ไหม, ดาวน์โหลด tokenizer ได้ไหม รันแล้วต้องเห็น ✓ ครบทุกบรรทัดก่อนไปต่อ
<code>train_qwen36.py</code>	สคริปต์เทรนตัวจริงที่ใช้ — โหลด <code>unsloth/Qwen3.6-35B-A3B</code> , ตั้งค่า LoRA (<code>LORA_R=32</code> , <code>lora_dropout=0</code> เพราะ MoE ParamWrapper ไม่รองรับ dropout≠0), เทรนแบบ bf16 (ห้าม 4-bit เพราะ Unsloth เตือนว่าตระกูลนี้ quantize แล้วคุณภาพตกผิดปกติ), <code>MAX_LENGTH=24576</code> (แก้จาก 9216 หลังเจอบั๊ก grid-master ตัดคำตอบทิ้ง — ตอนที่ 2 ข้อ 5), freeze vision layers (<code>finetune_vision_layers=False</code>) รองรับ <code>TEST_STEPS</code> (ทดสอบสั้นก่อนรันเต็ม)
<code>train_qwen3v1.py</code>	สคริปต์เทรนสำรอง สำหรับโมเดลตระกูล Qwen3-VL (8B/32B, 4-bit QLoRA) ใช้ได้บนการ์ดเล็กกว่า (4090 24GB พอ) — นี่คือนิสคริปต์ที่ใช้เทรน t02 จริง (ตอนที่ 3)
<code>eval_fields.py</code>	สคริปต์วัดผลระดับ field — เทียบคำตอบโมเดลกับ <code>val.jsonl</code> นับ JSON valid %, view match %, element recall, field exact-match รองรับ <code>--base</code> (วัด baseline ก่อนทุน), <code>--adapter <path></code> (วัดหลังทุน), <code>--limit N</code>
<code>export_gguf.py</code>	สคริปต์ export ครบวงจร — โหลดโมเดล+LoRA adapter → <code>merge_and_unload()</code> → เช็ค sanity (ดูว่า merge ใช้ได้จริงไหมก่อนเสียเวลา quantize — ด้านนี้เองที่ตอนที่ 4 พบว่าอ่อนเกินไปสำหรับ t01) → <code>save_pretrained_gguf(..., quantization_method="q4_k_m")</code> → เช็คขนาดไฟล์ → ดาวน์โหลด <code>mmpoj-F16.gguf</code> จากต้นฉบับ → อัปโหลดทุกไฟล์ขึ้น HuggingFace → verify ผ่าน API จริง
<code>stats.json</code>	สถิติ + ค่า config ที่ใช้ตอน build dataset ล่าสุด (จำนวน examples, การกระจาย pattern ฯลฯ) — ไว้ตรวจสอบย้อนหลังว่าตอน build ใช้ config อะไร
<code>finetune_results_2026-07-21.png</code>	กราฟผลการเทรนจริงของรอบ 2026-07-21 (เก็บไว้เป็นหลักฐาน/อ้างอิง)

A.3 ผล/ — ผลลัพธ์การถอดแบบบ้าน_เล็ก_1ชั้น_01 จริง (จาก `extract_house01_local.py`)

ไฟล์/โฟลเดอร์	หน้าที่
<code>_classify.json</code>	ผลลัพธ์ Phase A (classify) — pattern ของแต่ละ view ในแต่ละหน้า ใช้ตัดสินว่าหน้าไหนต้องเข้า Phase B (ถอดเต็ม)
<code>extract_log.txt</code>	log การรันทั้งหมด (ทั้ง Phase A และ B) ตามเวลาจริง
<code>llama_server_stdout.log</code>	log ดิบจากตัว <code>llama-server</code> เอง (ข้อความเตือนของ llama.cpp เช่นเรื่อง image tokens อยู่ ในนี้)
<code>บ้าน_เล็ก_1ชั้น_01_หน้าNN_ai.json</code>	ผลลัพธ์ Phase B (extract เต็ม) ของหน้า NN ที่ parse เป็น JSON สำเร็จ
<code>_raw/</code> <code>บ้าน_เล็ก_1ชั้น_01_หน้าNN.txt</code>	ข้อความดิบที่โมเดลตอบกลับมา (ก่อน parse) — เก็บไว้เพื่อ parse JSON ไม่ผ่าน จะได้ย้อนดูว่า โมเดลตอบอะไรมาจริงๆ

A.4 ai_output_บ้าน_ใหญ่_1ชั้น_01/ — ผลทดสอบ smoketest แยกต่างหาก

ไฟล์	หน้าที่
<code>_prompt_short.txt</code>	สำเนาของ prompt สักดจริง (<code>PROMPT_SHORT</code>) ที่บันทึกไว้ตอนรัน smoketest ครั้งนั้น — เนื้อหาเดียวกับที่อยู่ในภาคผนวก B ด้านล่าง
<code>_smoketest_หน้า10.log</code> / <code>_smoketest_หน้า10.out</code>	log และผลลัพธ์ดิบของการทดสอบรันหน้า 10 ของบ้านใหญ่ 1 ชั้น 01 (ทดสอบแยกจาก บ้านเล็ก 01 ที่เป็นชุดหลัก)

ภาคผนวก B — prompt จริงที่ส่งไปหา Qwen (t01)

ข้อควรรู้ก่อนอ่าน: Qwen ในภาคผนวกนี้ ไม่ใช่ Qwen-VL-Max ของ Alibaba DashScope ที่แอป Constistant (`js/ai/qwenVision.js` + Supabase Edge Function `qwen-vision`) เรียกผ่านคลาวด์ — อันนั้นเป็นโมเดลบริการออนไลน์คนละตัว คนละ pipeline คนละ prompt กัน (prompt ของฝั่งนั้นอยู่ที่ `js/ai/prompts.js` ใน repo `Constistant`) ส่วน Qwen ในเอกสารนี้คือ `Qwen3.6-35B-A3B` ที่มาแบบตัวเอง (t01) แล้วรันเป็นไฟล์ GGUF อยู่บนเครื่องมะขามเอง ผ่าน `llama-server` — คนละโมเดล คนละที่รัน คนละ prompt แต่มีเป้าหมายเดียวกัน (อ่านแบบก่อสร้างไทยแล้วสกัดข้อมูลเป็น JSON)

มี prompt จริง 2 ตัวที่ `extract_house01_local.py` ส่งให้โมเดล t01 (มาจาก 2-phase pipeline ที่อธิบายไว้ในตอนที่ 4 ข้อ 7 — extraction เต็มทุกหน้าเข้ากันไป ~18 นาที/หน้า เลยแยกเป็นสแกนไว้อีกก่อนแล้วถอดเฉพาะหน้าที่ต้องการ):

B.1 Phase A — `CLASSIFY_PROMPT` (สแกนไว้ออกแค่ pattern ของแต่ละ view)

ส่งพร้อมภาพ 1 หน้า ให้โมเดลตอบสั้นๆ (จำกัด `MAX_TOKENS_CLASSIFY = 300`) เพื่อรู้ว่าหน้านั้นมี view แบบไหนบ้าง ก่อนตัดสินใจว่าจะส่งเข้า Phase B (ถอดเต็ม) หรือข้าม:

```
You are looking at one page of a Thai reinforced-concrete construction drawing set.
Identify every distinct view/box on this page. Reply with ONLY a JSON array of pattern
strings, one entry per view (same order as they appear on the page), using exactly one
of these values per entry: plan, section, schedule, notes, index, material_list,
site_plan, side_profile, gridline, title, symbol, roof_plan, misc, unknown.
Example: ["plan", "schedule"]
No commentary, no markdown fence, JSON array only.
```

เฉพาะหน้าที่ classify แล้วมี pattern อยู่ใน `KEEP_PATTERNS = {"plan", "section", "roof_plan", "side_profile", "gridline"}` เท่านั้นที่จะถูกส่งเข้า Phase B — หน้าที่เหลือ (schedule, notes, index, material_list, site_plan, title, symbol, misc, unknown) จะถูกข้าม ประหยัดเวลาไปมาก

B.2 Phase B — `PROMPT_SHORT` (ถอดเต็ม — ตัวเดียวกับที่ใช้สอนตอนเทรน)

นี่คือ prompt หลักของทั้งโปรเจกต์ — สำคัญที่สุดคือมัน "byte-identical" กันทั้ง 3 ที่: instruction ที่สอนโมเดลตอนเทรน (`PROMPT_SHORT` ใน `build_dataset.js`), instruction ที่ใช้วัดผล (`eval_fields.py`), และ instruction ที่ใช้ตอนถอดแบบจริง (`extract_house01_local.py` — คอมเมนต์ในไฟล์เขียนกำกับไว้เองว่า "byte-identical to PROMPT_SHORT in `tune_ai/t01/data_before_tune/build_dataset.js`") ความเหมือนกันแบบนี้คือสิ่งที่ต้องรักษาไว้เสมอ — สอนอย่างหนึ่งแล้วถามอีกอย่างหนึ่ง (train/inference prompt mismatch) จะทำให้ผลลัพธ์ที่เคยได้ไม่มีความหมาย เหมือนกับปัญหา "ความละเอียดภาพตอนเทรน vs ตอนใช้ต่างกัน 14 เท่า" ที่ตอนที่ 4 เจอ (เป็นปัญหาคคนละจุดกัน — อันนั้นคือ "ภาพ" ไม่ตรงกัน อันนี้คือ "คำสั่ง" ต้องตรงกัน) — ส่งพร้อมภาพ 1 หน้า จำกัดคำตอบยาวสุด `MAX_TOKENS_EXTRACT = 9000`:

```
You are reading one page of a Thai reinforced-concrete (RC) construction drawing set.
Extract everything on the page into JSON following the primary_rawjson_schema.

Inventory EVERY view/box on the page first, then emit one entry per view in "views"
(a single-view page still uses a one-entry array — never drop a view). Each view
carries its own "pattern": plan, section, schedule, notes, index, material_list,
site_plan, side_profile, gridline, title, symbol, roof_plan, misc, or unknown.

GRID AND DUMMY GRID — the single most error-prone part of this task, read carefully:
- grid_ref reads row-letter first, then column ("A-1", not "1-A"). Point-type elements
  (footing/column) use a grid_refs array instead of start/end.
- A structural line not on a named/printed grid still needs a name: append a prime to
  the nearest named grid ("1'", "A'"). If more than one dummy line falls in the same
  gap, number them in reading order (left→right / top→bottom): 1st gets one prime,
  2nd gets two.
```

- THE KEY RULE: if a beam's start or end point does not sit on any grid line you can see, that point still needs a grid line — it does NOT mean the beam should be dropped. Trace every beam segment, including short stubs near stairs/closets. For each endpoint: use the existing named/dummy grid if one is there; if not, read its position off a printed dimension chain and record a new dummy grid, then reference the beam against it. Never: (a) drop the beam because it "isn't on the grid", (b) write a prose description instead of grid_ref_start/grid_ref_end, (c) set start=end with a null span. Exception: a slab/eave edge with no beam label and no corner columns is not structural and needs no dummy grid.
- Span length comes from the grid table, not your own visual estimate.

REBAR (main_bar):

- Always split top/bottom, even when the counts are equal — never collapse into one.
- If a section shows a clearly distinct row of bars at mid-depth (own leader line, sitting between the top and bottom rows, usually a deep beam), record it as a third face, main_bar.middle. Do not fold it into additional_bars, and do not invent one by splitting a top/bottom cluster.
- A circle symbol (Ø) always means round bar (RB); visible ribs mean deformed bar (DB) — read the symbol, never infer type from diameter.
- Columns use a single main_bar.count for the 4 corner bars — do not split top/bottom.
- Before assigning an "additional" bar to top or bottom, check the leader line itself, not just the label wording — the same-looking label has resolved to opposite sides on different marks in this series.

OUTPUT DISCIPLINE:

- Same element_id appearing more than once on this page with non-overlapping positions → merge into one entry (sum count, concatenate grid_refs). Exception: a multi-level schedule keeps the same element_id per level as separate entries, using a "level" field — never embed the level into element_id.
- One atomic entry per grid-to-grid beam segment; do not pre-group same-mark spans.
- Reading order: top-to-bottom by row, left-to-right by column, vertical before horizontal at a shared start point.
- Use null for anything unclear. Do not guess or invent a value.

Reply with JSON only. No markdown fence, no commentary.

ทำไม instruction หน้าตาแบบนี้ ไม่ใช่แบบอื่น: ไม่ได้เขียนแบบสเปคเต็ม (~6,000 token) และไม่ได้ใช้ตัวสันสรุปเท่ากันทุกกฎ — วัดจาก `workmen's_diary/` จริง (grep-count 2026-07-20) ว่าแต่ละหัวข้อถูกแก้/พูดถึงกี่ครั้งใน 3 อาทิตย์ แล้วให้พื้นที่ใน prompt ตามความถี่นั้น: เรื่อง dummy grid + beam-endpoint (พูดถึง 48 ครั้ง) ได้พื้นที่ยาวสุด, เรื่อง main_bar top/bottom (50 ครั้งรวมกัน) ได้พื้นที่สูง, ส่วน confidence/price-table/specs join (≤16 ครั้ง) ตัดออกทั้งหมดเพราะ dataset ไม่มีให้สอนอยู่แล้ว — รายละเอียดเต็มอยู่ใน `tune_ai/t01/data_before_tune/README.md` หัวข้อ "instruction ~1,020 tok"

⚠ **เกี่ยวข้องกับตอนที่ 4 โดยตรง:** prompt นี้เองที่เป็นต้นเหตุที่ทำให้ด้านตรวจ `export_gguf.py:107` (เห็นว่ามีคำว่า `views/pattern` ในคำตอบใหม่) ใช้แยกแยะ "โมเดลทุนแล้ว" กับ "โมเดลเปล่าที่แค่ทำตาม prompt" ไม่ได้ — เพราะคำว่า `views` และ `pattern` ถูกบอกรัวไว้ใน prompt เองจริงๆ โมเดลไหนก็ตอบได้แม้ไม่เคยทุนเลย (ดูตอนที่ 4 ข้อ 4 หลักฐาน B) นี่คือเหตุผลที่ด้านตรวจรอบต่อไปต้องเช็คค่าที่ **ไม่ได้อยู่ใน prompt** (เช่น `png/doc_page` ที่มีอยู่แค่ในเฉลย) แทน